Physical AI 실무 · 1일차

ALL AI

Jetson·Ubuntu·Python·C++ 기초

v2

v2 · Orin Nano / JetPack 6.2 초기 설정 부록 추가

경기도 성남시 분당구 판교로 289번길 20 판교테크노밸리
스타트업캠퍼스 1동 806호-807호

Copyright @ 2026 ALL.AI Inc. All rights reserved.

www.allai.co.kr

수업 시작 전 · 압축 해제 → 필수 설치 → 이미지 load

1일차 · 시작 준비 | 인터넷에 연결된 Jetson 호스트에서 로컬 담당자 한 명이 최초 한 번 실행

```
# 1. 배포 파일 압축 해제
# 기존 ~/NEPES가 없는 장비
cd ~
tar -xzf NEPES-v1.0.tar.gz
cd ~/NEPES

# 2. jtop·curl 설치
bash 01.Setup/01.jtop/install_jtop.sh
sudo apt-get install -y curl

# 다음: 오른쪽 Docker 01 → 02
# Jupyter가 아닌 호스트에서 실행
```

```
# 3. Docker 01 → NVIDIA 런타임 02
cd ~/NEPES/01.Setup/02.Docker
bash 01_install_docker.sh
bash 02_setup_nvidia_runtime.sh

# 4. 로그아웃 후 재로그인
# SSH 사용자는 연결 종료 후 재접속
# 새 호스트 터미널에서 계속
cd ~/NEPES
docker load -i \
  01.Setup/03.Image/v1.0.tar
docker ps -a
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

로컬 사용자 · 오늘의 실습 환경 생성

1일차 · 시작 준비 | USB 연결 → Jetson 데스크톱의 호스트 터미널

- USB 카메라를 먼저 연결하고 Jetson 데스크톱에 로그인합니다.
- 본인 컨테이너가 없을 때 오른쪽 명령으로 생성합니다.
- 접속: http://localhost:8888
- 로그인 토큰: nvidia
- 매일 같은 이름·폴더·포트·ROS Domain을 사용합니다.

```
cd /home/nvidia/NEPES
cd 01.Setup/02.Docker
ROS_DOMAIN_ID=21 \
  bash 04_run_container.sh \
  nepes-course:physical-ai-v1.0 \
  /home/nvidia/NEPES \
  local physical_ai_local 8888
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

원격 사용자 · 개인 자료와 실습 환경

1일차 · 시작 준비 | PC에서 SSH 접속 후 Jetson 호스트에서 실행

- 개인 폴더 복사는 첫날 한 번만 수행합니다. 이미 있으면 복사 명령을 건너뜁니다.
- 재부팅 후에는 같은 개인 폴더로 컨테이너를 생성합니다.
- PC 접속: http://<Jetson-IP>:8889
- 토큰: nvidia · 독립 ROS Domain: 22
- USB 실습은 로컬 화면을 함께 봅니다.

```
ssh nvidia@<Jetson-IP>
cd /home/nvidia/NEPES
# 개인 폴더가 없을 때만 복사
bash 01.Setup/02.Docker/\
07_copy_workspace.sh \
  /home/nvidia/NEPES \
  /home/nvidia/NEPES_remote01
# 매일: 개인 컨테이너가 없을 때 생성
cd 01.Setup/02.Docker
ROS_DOMAIN_ID=22 bash 04_run_container.sh \
  nepes-course:physical-ai-v1.0 \
  /home/nvidia/NEPES_remote01 \
  remote physical_ai_remote01 8889
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

준비 완료 · Jupyter에서 저장과 커널 확인

1일차 · 시작 준비 | 여기부터 개인 컨테이너의 **Jupyter Terminal**

- 파일 목록에 02.Python·03.C++·Data가 보여야 합니다.
- Python 경로는 /opt/course/bin/python입니다.
- 노트북은 첫 준비 셀부터 실행합니다. 다른 노트북의 변수는 공유되지 않습니다.
- 매일 소스·결과를 저장하고 종료합니다. 재부팅 후 파일은 남고 변수·프로세스는 사라집니다.

```
cd /workspace
pwd
which python
echo "$COURSE_MODE"
mkdir -p Data/outputs
printf "ready\n" > \
  Data/outputs/day1-ready.txt
cat Data/outputs/day1-ready.txt
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

오늘의 도착점: 직접 실행하고 결과를 설명하기

1일차 · 시작과 전체 지도 | 도입 30분

- Jetson의 실행 위치와 파일 저장 위치를 구분합니다.
- Ubuntu 명령으로 자료를 찾고 개인 실습 환경에 접속합니다.
- Python으로 센서 보고서를 만들고 C++ 계산 함수를 Python에서 호출합니다.
- 함께 해석하기: 입력·처리·결과를 자기 말로 설명하고 수정한 코드와 결과를 저장해야 합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

8시간의 학습 흐름

1일차 · 시작과 전체 지도 | 도입 30분

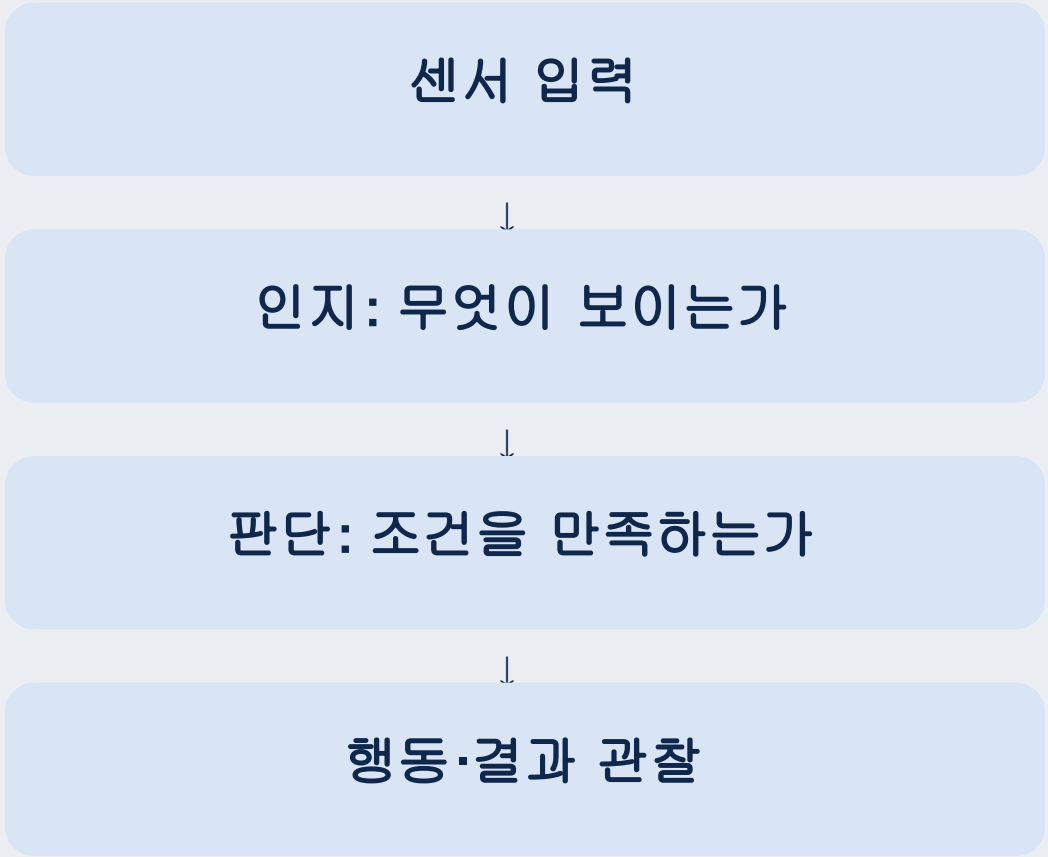
- Physical AI·Jetson 소개 30분 → Ubuntu 기초 30분 → 환경 구성·Jupyter 60분.
- Python 기본 문법·자료구조 120분 → C++ 기초·빌드·Python 연동 240분.
- 실습과 정리 시간을 포함한 480분이며 휴게·식사는 별도입니다.
- 함께 해석하기: 본수업 시간에는 지정된 핵심 TODO·검증·결과 해석을 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Physical AI: 인지·판단·행동의 연결

1일차 · 시작과 전체 지도 | 도입 30분

- 인지는 카메라 등 센서에서 관측을 만듭니다.
- 판단은 관측과 작업 규칙을 비교합니다.
- 행동은 허용된 명령을 실행하고 결과를 다시 관찰합니다.
- 함께 해석하기: 인지는 보이는 정보를 만들고 판단은 작업 조건과 대조하며 행동은 검증된 결과를 사용합니다.



본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

5일 동안 하나의 작업대 사례를 발전시키기

1일차 · 시작과 전체 지도 | 도입 30분

- 1일차: 센서 값을 계산하고 파일로 남깁니다.
- 2일차: 글과 이미지에 질문하고 응답을 검증합니다.
- 3일차: 영상 처리 결과를 ROS 메시지로 전달합니다.
- 4일차: 사람이 설계한 Pseudo-VLA로 손 추종·정지를 평가합니다.
- 5일차: MuJoCo·LIBERO에서 학습된 SmolVLA 정책을 실행합니다.
- 함께 해석하기: 상황을 매번 새로 배우는 부담을 줄이고 입력 표현·판단 방식·행동 연결이 어떻게 달라지는지 보기 위해서입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Jetson은 CPU와 GPU를 갖춘 소형 컴퓨터

- CPU는 프로그램 흐름·파일·네트워크·입출력을 처리합니다.
- GPU는 모델의 많은 계산을 병렬로 수행합니다.
- 메모리와 저장장치는 역할이 다르며, 모든 프로그램이 자동으로 GPU를 사용하는 것은 아닙니다.

카메라·네트워크 입력

CPU: 프로그램·입출력

GPU: 모델 계산

메모리 ↔ 저장 파일

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

모듈·캐리어보드·주변장치 구분

1일차 · 시작과 전체 지도 | 도입 30분

- Jetson 모듈은 주요 계산 장치를 담고, 캐리어보드는 전원·USB·화면·네트워크 연결을 제공합니다.
- 교육 장비는 커스텀 보드이므로 포트 위치·전원 규격은 강사 실물 안내를 따릅니다.
- 전원·화면·키보드·네트워크를 확인한 뒤 USB 카메라를 연결합니다.

전원·냉각: 실물 확인

Jetson 모듈 + 캐리어보드

USB: 키보드·카메라

화면·네트워크 연결

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

터미널은 명령을 전달하는 창입니다

- 앞의 시작 준비로 연 개인 컨테이너의 Jupyter Terminal에서 연습합니다. 환경 구성의 원리는 다음 절에서 복습합니다.
- 명령어 + 옵션 + 대상 순서로 읽습니다. Enter로 실행하며 안내의 $ 기호는 입력하지 않습니다.
- 대소문자와 공백은 의미가 있으므로 명령과 경로를 확인합니다.

```
pwd
ls
ls -l Data
# $ 기호 없이 명령만 입력
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

현재 위치와 절대·상대 경로

1일차 · Ubuntu 기초 | 따라 해보기 30분

- pwd는 현재 폴더를 출력합니다. 상대 경로는 현재 위치를 기준으로 해석됩니다.
- /로 시작하면 절대 경로입니다. ..는 상위 폴더, .은 현재 폴더입니다.
- Data와 data는 다른 이름입니다. 컨테이너의 교재 시작 위치는 /workspace입니다.
- 함께 해석하기: 상대 경로 Data는 현재 폴더에 따라 달라집니다.

```
pwd
cd /workspace
ls Data
cd 02.Python
pwd
cd ..
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

파일·폴더 목록을 읽기

1일차 · Ubuntu 기초 | 따라 해보기 30분

- ls는 이름을, ls -l은 권한·소유자·크기 같은 상세 정보를 보여줍니다.
- ls -a는 점으로 시작하는 숨김 항목까지 표시합니다.
- ~는 현재 사용자의 홈입니다. 호스트와 컨테이너에서 홈 경로가 다를 수 있습니다.
- 경로 일부를 입력한 뒤 Tab으로 자동완성하고 ↑로 이전 명령을 불러옵니다.

```
ls -l /workspace
ls -a /workspace
echo ~
whoami
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

폴더 만들기·복사·이름 바꾸기

1일차 · Ubuntu 기초 | 따라 해보기 30분

- mkdir -p는 필요한 상위 폴더까지 만듭니다.
- cp는 원본을 남기고 복사합니다. mv는 이동 또는 이름 변경입니다.
- 결과는 명령 메시지뿐 아니라 ls와 파일 내용으로 다시 확인합니다.
- 함께 해석하기: 일반 파일 복사는 별도 파일을 만듭니다.

```
cd /workspace
mkdir -p Data/outputs/ubuntu-practice
cd Data/outputs/ubuntu-practice
echo hello > note.txt
cp note.txt backup.txt
mv backup.txt saved.txt
ls -l
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

삭제·덮어쓰기는 대상을 먼저 확인하기

- rm은 지정 파일을 삭제합니다. 연습용 파일만 대상으로 사용합니다.
- >는 파일을 덮어쓰고 >>는 끝에 추가합니다.
- 명령을 실행하기 전에 pwd와 ls로 위치와 이름을 확인합니다.
- 함께 해석하기: 현재 위치와 실제 대상 경로를 목록으로 확인합니다.

```
pwd
ls -l
cat saved.txt
rm saved.txt
echo second >> note.txt
cat note.txt
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

파일 내용과 긴 로그 읽기

ALL AI

- cat은 짧은 파일 전체를 표시합니다. less는 긴 파일을 이동하며 읽고 q로 나옵니다.
- head와 tail은 앞부분과 뒷부분을 확인합니다.
- 오류 로그의 마지막 부분을 읽되 Python과 C++에서 원인 위치를 찾는 방식은 구분합니다.

```
cat note.txt
less /workspace/학습안내.md
head -n 5 note.txt
tail -n 5 note.txt
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

검색과 파이프로 출력 좁히기

1일차 · Ubuntu 기초 | 따라 해보기 30분

- grep은 문자열이 있는 줄을 찾습니다. -n을 붙이면 줄 번호가 보입니다.
- 파이프 |는 앞 명령의 출력을 뒤 명령의 입력으로 전달합니다.
- 로그 전체를 복사하기보다 오류 이름·자기 파일 위치·직전 명령을 함께 확인합니다.
- 함께 해석하기: 보통 앞 명령의 출력을 뒤 명령의 입력으로 넘깁니다.

```
cd /workspace
grep -n "커널" 학습안내.md
ls Data | head -n 5
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

프로그램 실행과 실행 위치

- bash script.sh는 셸 스크립트, python file.py는 Python 프로그램을 실행합니다.
- Jupyter Python 셀과 터미널은 서로 다른 문법을 입력하는 곳입니다.
- 같은 파일명이라도 현재 경로와 실제 Python 실행 파일을 확인해야 합니다.
- 함께 해석하기: 프로그램의 상대 경로가 현재 디렉터리를 기준으로 해석되기 때문입니다.

```
pwd
which python
python --version
# Python 파일은 터미널에서 실행
# python 프로그램.py
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

환경변수와 source의 범위

- export는 현재 셸과 이후 시작하는 자식 프로세스에 설정을 전달합니다.
- source는 스크립트 내용을 현재 셸에 적용합니다. ROS 환경 준비에서 사용합니다.
- 다른 터미널은 별도 환경일 수 있으므로 새 창에서 설정을 다시 확인합니다.
- 함께 해석하기: 새 터미널에는 자동으로 전달되지 않습니다.

```
echo "$COURSE_MODE"
echo "$ROS_DOMAIN_ID"
export PRACTICE_LABEL=day1
echo "$PRACTICE_LABEL"
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Ctrl+C·exit·창 닫기의 차이

- Ctrl+C는 터미널의 현재 실행 프로그램에 중단을 요청합니다.
- exit는 현재 셸을 나갑니다. docker exec로 연 셸에서 나와도 주 컨테이너는 계속 실행될 수 있습니다.
- 브라우저 창을 닫는 것과 커널·서버를 종료하는 것은 다릅니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

메모리·저장 공간·프로세스 확인

1일차 · Ubuntu 기초 | 따라 해보기 30분

- free -h는 메모리, df -h는 파일시스템 저장 공간을 확인합니다.
- ps·top은 실행 프로세스, jtop은 Jetson 자원 상태를 살펴보는 도구입니다.
- 강사 호스트에서 모델 로드 전·중·후의 메모리·부하를 비교합니다. 자원은 같은 장비에서 공유합니다.

```
free -h
df -h /workspace
ps -ef | head
# Jetson 호스트에서 강사 시연
# jtop
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Ubuntu 미니 실습 완료 확인

1일차 · Ubuntu 기초 | 따라 해보기 30분

- 개인 연습 폴더를 만들고 파일을 생성·복사·이름 변경합니다.
- cat으로 내용을 확인하고 연습용 복사본 하나만 삭제합니다.
- 현재 경로·남은 파일·사용한 명령의 의미를 옆 사람에게 설명합니다.
- 함께 해석하기: 현재 컨테이너 경로, 만든 파일의 위치, 저장 후 다시 읽은 내용으로 설명합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

개인 PC·Jetson·컨테이너의 관계

- 개인 PC의 브라우저는 화면을 보여주고 실제 실습 코드는 Jetson의 컨테이너에서 실행됩니다.
- Jetson 호스트는 Docker와 장치 연결을 관리합니다.
- Jupyter 터미널의 /workspace는 개인 교재 폴더를 연결한 경로입니다.
- 함께 해석하기: 브라우저는 Jetson의 Jupyter에 접속하는 화면입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Docker 이미지와 컨테이너

1일차 · 로컬·원격 환경 구성 | 환경·Jupyter 60분

- 이미지는 준비된 프로그램 환경이고 컨테이너는 그 환경으로 시작한 실행 공간입니다.
- 한 이미지로 여러 개인 컨테이너를 만들 수 있습니다.
- 교재·Data는 바깥 압축을 풀어 배치하고, 내부 이미지 tar는 docker load로 읽습니다.
- 함께 해석하기: load는 실행 환경 이미지를 Docker에 등록합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

누가 무엇을 준비하는가

1일차 · 로컬·원격 환경 구성 | 환경·Jupyter 60분

- 로컬 담당자: OS·네트워크·저장 공간, Docker·NVIDIA 런타임, 배포 이미지와 모델을 준비합니다.
- 수강생: 개인 폴더·컨테이너·포트와 Jupyter 접속을 확인합니다.
- 공용 Ollama 서버는 같은 Jetson의 로컬 사용자 한 명만 생성·실행합니다.
- 나머지 로컬·원격 사용자는 서버를 따로 만들지 않고 공용 API를 공유합니다.
- 함께 해석하기: 로컬 담당자가 보드의 공용 서버를 준비하고 학생은 배정된 주소를 사용합니다.

```
# Jetson 호스트 터미널: 로컬 담당자 한 명만 실행
cd /home/nvidia/NEPES
bash 01.Setup/02.Docker/06_run_ollama.sh \
  /home/nvidia/NEPES
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

로컬 사용자: 장비와 교재 확인

- Jetson 데스크톱에서 터미널을 열고 호스트 교재 폴더로 이동합니다.
- USB 카메라는 컨테이너 생성 전에 연결합니다. 로컬 모드는 실제 Jetson 화면의 DISPLAY를 사용합니다.
- 강사가 준비한 이미지와 Data를 확인한 뒤 개인 컨테이너를 만듭니다.

```
cd /home/nvidia/NEPES
pwd
ls 01.Setup
docker image ls
ls /dev/video*
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

로컬 컨테이너 생성 명령 읽기

- 인자는 이미지·호스트 교재 폴더·모드·컨테이너 이름·포트 순서입니다.
- local 모드는 카메라 장치와 로컬 화면 연결을 준비합니다.
- 앞 3쪽에서 생성했다면 상태·인자만 확인합니다. 없을 때만 실행합니다.
- 21은 예시입니다. 독립 실습에서는 로컬·원격 모두 수강생별로 다른 Domain ID를 사용합니다.

```
cd /home/nvidia/NEPES
cd 01.Setup/02.Docker
ROS_DOMAIN_ID=21 \
  bash 04_run_container.sh \
  nepes-course:physical-ai-v1.0 \
  /home/nvidia/NEPES \
  local physical_ai_local 8888
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

로컬 접속과 성공 확인

- Jetson 브라우저에서 http://localhost:8888을 엽니다. 교육용 공통 토큰은 nvidia입니다.
- Jupyter 파일 목록에서 02.Python·03.C++와 Data를 확인합니다.
- 터미널에서 /workspace와 Python 경로를 확인하고 첫 환경 셀을 실행합니다.

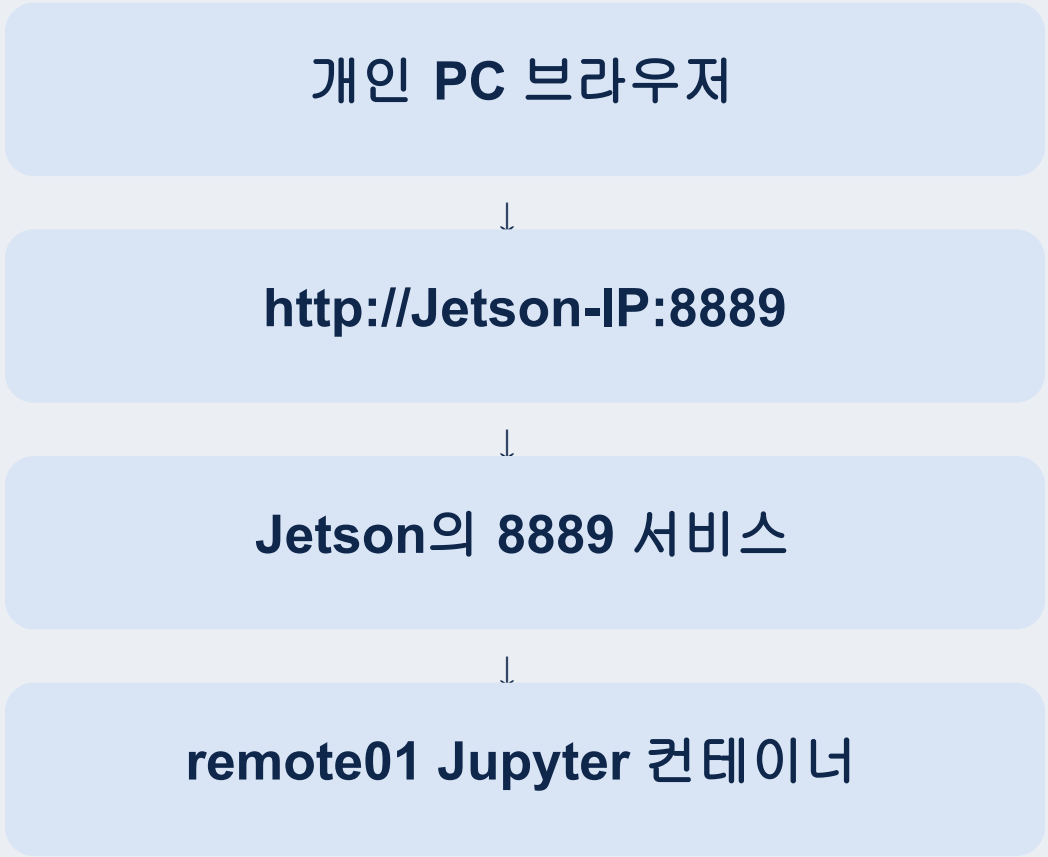
```
# 개인 컨테이너의 Jupyter Terminal
pwd
which python
ls /workspace
echo "$COURSE_MODE"
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

원격 접속: IP·포트·localhost

1일차 · 로컬·원격 환경 구성 | 환경·Jupyter 60분

- IP는 접속할 장비를, 포트는 그 장비의 서비스 진입점을 구분합니다.
- 개인 PC의 localhost는 그 PC입니다. 원격 Jetson을 가리키지 않습니다.
- 교재 기본 경로는 개인 PC 브라우저에서 Jetson-IP:할당포트로 접속하는 방식입니다.
- 함께 해석하기: localhost는 브라우저가 실행되는 그 PC입니다.

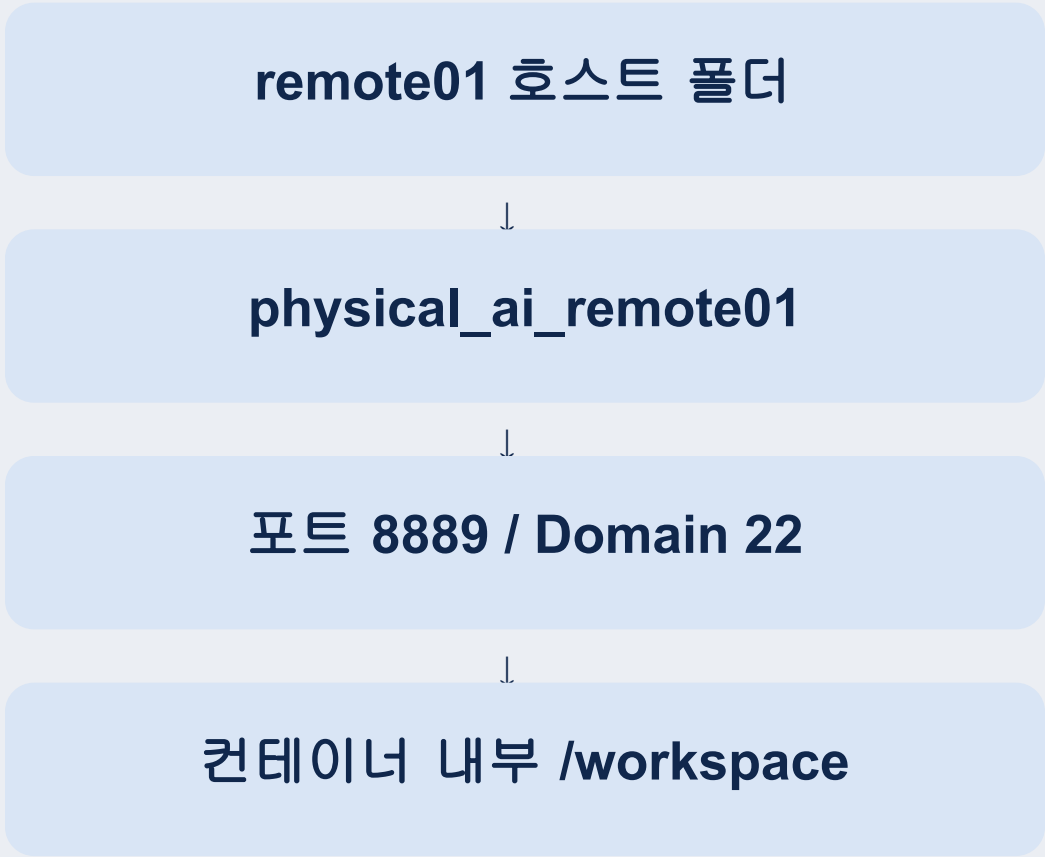


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

원격 사용자도 Jetson 위에 개인 폴더를 둡니다

ALL AI

- 개인 자료는 NEPES_remote01처럼 호스트의 별도 폴더로 준비합니다.
- 컨테이너 이름·포트·ROS_DOMAIN_ID를 사용자별로 구분합니다.
- 내부 경로는 모두 /workspace로 같아도 연결된 호스트 폴더는 서로 다릅니다.



본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

원격용 자료 복사: 무엇을 포함하는가

- 개인 교재·입력 데이터·참고 가중치를 복사합니다.
- 5일차 tokenizer 캐시·LIBERO 설정과 자산도 포함합니다.
- 개인 결과·인증 정보·공용 Ollama 저장소는 복제하지 않습니다.
- 제공된 07_copy_workspace.sh를 사용합니다.
- 함께 해석하기: 실행 소스와 필요한 Data·모델·캐시도 배포 안내대로 있어야 합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

원격 컨테이너 생성

- 앞 4쪽에서 생성했다면 상태만 확인합니다. 없을 때만 호스트에서 실행합니다.
- remote 모드는 로컬 GUI·USB 카메라를 전달하지 않습니다.
- 예시 이름·폴더·포트·Domain은 본인 배정값으로 바꿉니다.
- 함께 해석하기: 같은 호스트에서 충돌할 수 있습니다.

```
cd /home/nvidia/NEPES
cd 01.Setup/02.Docker
ROS_DOMAIN_ID=22 bash \
  04_run_container.sh \
  nepes-course:physical-ai-v1.0 \
  /home/nvidia/NEPES_remote01 \
  remote physical_ai_remote01 8889
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

원격 Jupyter 접속과 입력 선택

- 개인 PC에서 http://Jetson-IP:8889와 토큰 nvidia로 접속합니다.
- 개인 노트북·터미널은 Jetson의 원격 컨테이너에서 실행됩니다.
- USB 카메라 실습은 두 수강생이 로컬 화면을 함께 봅니다.
- 역할을 교대하고 각자 관찰·판단을 기록합니다.

```
# 원격 Jupyter Terminal
pwd
echo "$COURSE_MODE"
echo "$COURSE_HEADLESS"
echo "$ROS_DOMAIN_ID"
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

로컬·원격의 공통점과 차이

- 공통: 같은 교재·내부 경로와 GPU·모델 서버를 사용합니다.
- 개인: 폴더·컨테이너·포트·ROS Domain을 구분합니다.
- USB 카메라: 로컬에서 한 번 실행하고 같은 화면을 함께 봅니다.
- 무거운 훈련·추론은 순서를 정하고 기록은 각자 보존합니다.
- 함께 해석하기: 교재 함수와 저장 규칙은 같고 접속 주소·개인 폴더·포트·Domain과 실물 화면 관찰 방식이 다릅니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

컨테이너 상태 확인과 추가 셸

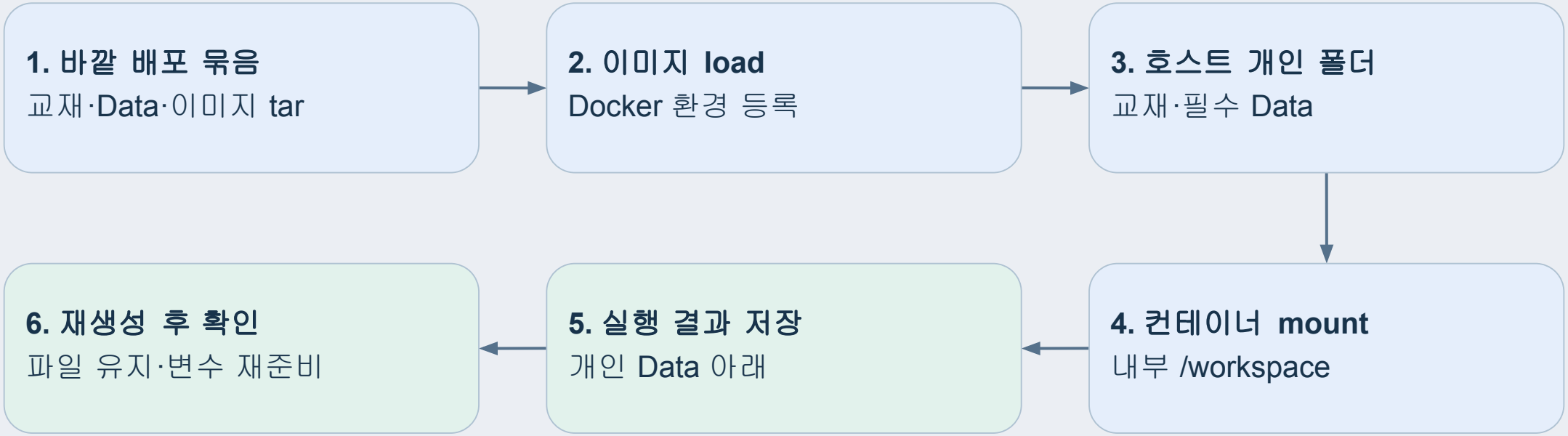
- docker ps는 실행 중인 컨테이너를 보여줍니다.
- docker exec는 이미 실행 중인 컨테이너에 추가 명령·셸을 엽니다.
- 추가 셸에서 exit해도 Jupyter 서버가 주 프로세스로 실행 중이면 컨테이너는 계속 유지됩니다.
- 함께 해석하기: 추가 셸은 같은 실행 환경에 명령 창을 하나 더 여는 것입니다.

```
# Jetson 호스트 터미널
docker ps
docker exec -it physical_ai_local \
  /bin/bash
# 이제 컨테이너 셸
pwd
exit
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

저장 파일과 실행 상태는 다릅니다

이미지와 호스트 Data는 따로 보존

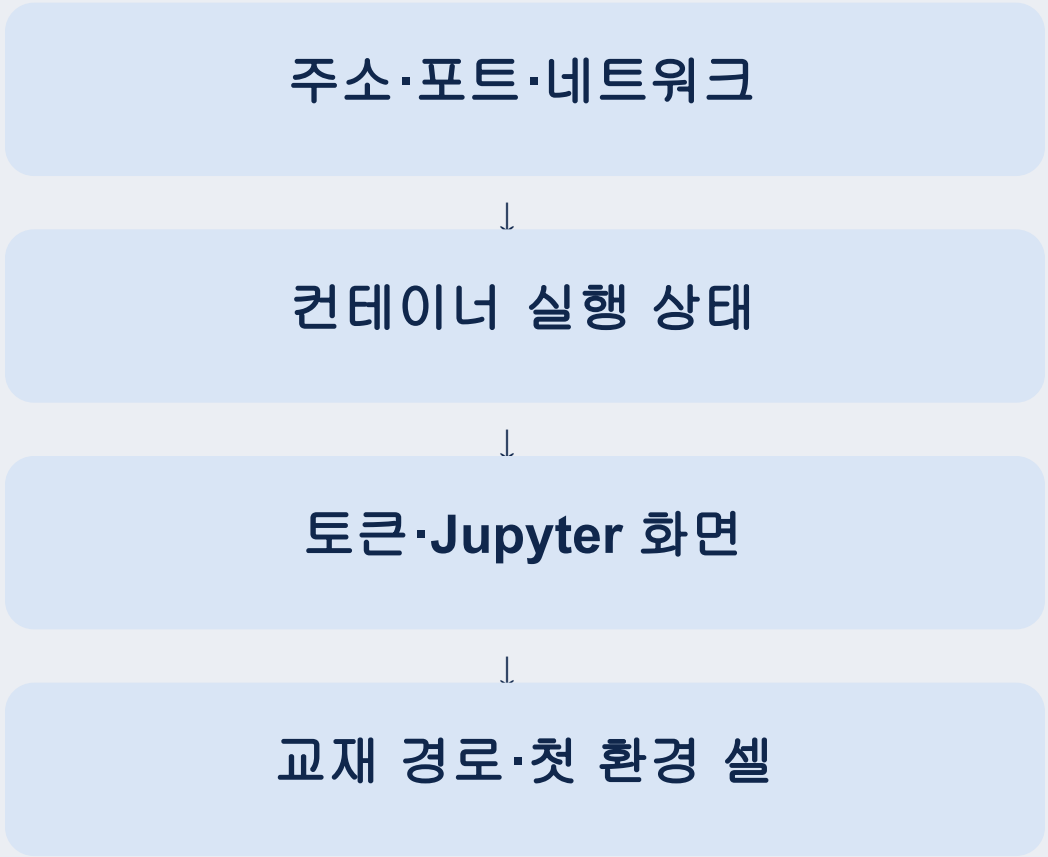


이미지 save에 bind mount Data가 포함되는 것은 아닙니다. 배포 묶음의 Data와 이미지를 각각 준비합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

접속이 안 될 때 확인할 순서

- 주소의 Jetson IP와 개인 포트를 확인합니다. localhost를 잘못 쓰지 않았는지 봅니다.
- 호스트에서 docker ps로 내 컨테이너가 실행 중인지 확인합니다.
- 파일을 못 찾으면 컨테이너의 pwd·ls, GPU 문제면 공용 작업 상태를 확인합니다.



본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Jupyter에서 네 가지 화면 구분

- Markdown 셀: 설명을 읽는 곳. Python 셀: 커널에 코드를 보내는 곳.
- 파일 편집기: .py·.cpp를 수정하고 저장하는 곳. Terminal: 독립 프로그램을 실행하는 곳.
- Shift+Enter는 셀 실행, Ctrl+S는 파일 저장입니다. 실행과 저장은 별개입니다.
- 함께 해석하기: 터미널은 셸 명령, 코드 셀은 선택한 커널의 코드를 실행합니다. 파일 편집기는 수정·저장, 출력 영역은 실행 결과입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

커널과 셀 실행 순서

1일차 · Jupyter 사용 | 환경 구성 마무리

- 커널은 앞서 실행한 변수·함수를 기억하는 Python 프로세스입니다.
- 셀을 수정만 하고 실행하지 않으면 이전 함수가 메모리에 남습니다.
- 상태가 헷갈리면 저장→커널 재시작→첫 환경 셀부터 순서대로 실행합니다.
- 함께 해석하기: 변수를 정의한 설정·import 셀이 실행됐는지 먼저 봅니다.

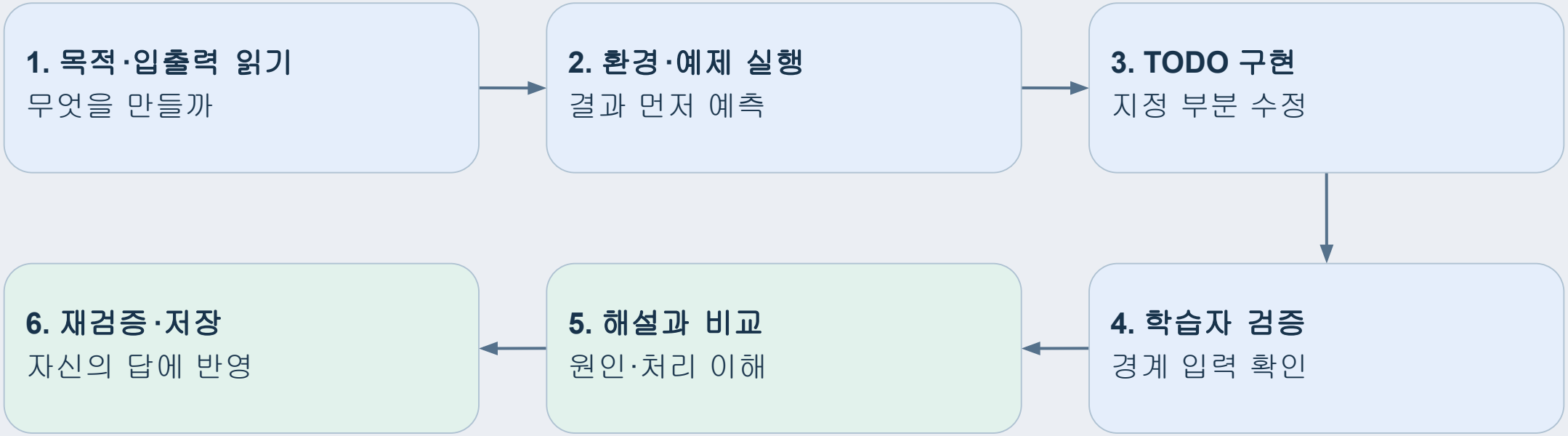
```
# Python 셀 1
temperature = 25

# Python 셀 2
print(temperature)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

TODO와 검증 셀의 역할

1일차 · Jupyter 사용 | 환경 구성 마무리

설명에서 TODO·검증·저장까지



검증 실패 시 TODO로 돌아갑니다. 정답 실행만으로 자신의 구현이 완성되지는 않습니다.

먼저 실습 | 02.Python/01_python_basics.ipynb

수업 핵심 1~4절과 완료 점검 확인 후 다음 슬라이드로 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Python: 입력·처리·출력으로 읽기

결과 복습 | 1일차 · Python 기본 문법 | Python 1교시 60분

- 프로그래밍은 입력을 받아 규칙에 따라 처리하고 결과를 돌려주는 절차입니다.
- 실습은 변수→조건→반복→함수 순서로 작은 계산을 조립합니다.
- 먼저 값의 타입과 예상 결과를 말하고 코드를 실행합니다.
- 함께 해석하기: 예를 들어 금액과 회원 여부가 입력, 할인 조건이 처리, 결제액이 출력입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

변수와 대입: 이름에 값을 연결하기

- =는 같은지 비교하는 기호가 아니라 오른쪽 결과를 왼쪽 이름에 연결하는 대입입니다.
- 같은 이름에 다시 대입하면 이후에는 새 값을 읽습니다.
- 변수 이름은 의미를 드러내고 대소문자를 구분합니다.
- 함께 해석하기: 이 숫자 예제에서 b는 3입니다.

```
count = 1
count = count + 1
print(count)
# 예상: 2
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

타입: 같은 모양도 다른 값입니다

- int는 정수, float는 실수, str은 문자열, bool은 참·거짓입니다.
- 문자열 "30"과 숫자 30은 연산 규칙이 다릅니다.
- 모르는 값을 받으면 type으로 확인하고 필요한 변환을 명시합니다.

```
raw = "30"
value = float(raw)
print(type(raw))
print(type(value))
print(value + 2)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

산술과 비교 연산

- +·-·*·/는 계산, ==·>=·<는 비교입니다.
- 비교 결과는 True 또는 False이며 조건문의 판단값으로 사용합니다.
- 평균 계산은 합계뿐 아니라 개수와 빈 입력 처리도 필요합니다.
- 함께 해석하기: 대입은 이름에 값을 연결하고 비교는 두 값의 관계를 bool로 계산합니다.

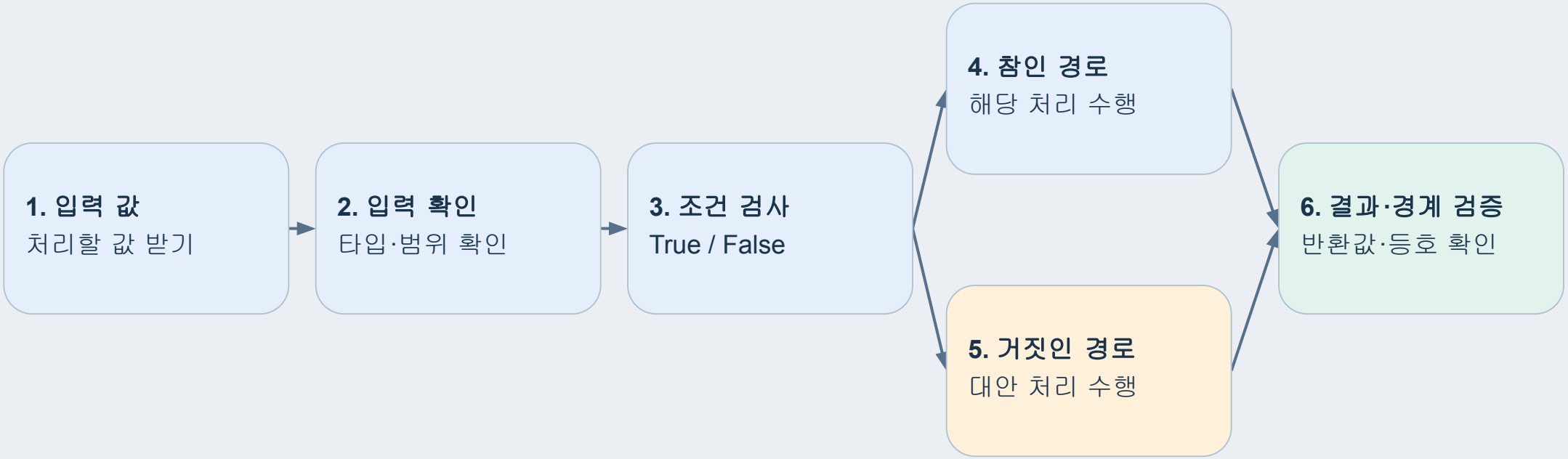
```
values = [20, 30, 25]
mean = sum(values) / len(values)
print(mean)
print(mean >= 30)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

조건문: 경계에서 경로가 바뀝니다

결과 복습 | 1일차 · Python 기본 문법 | Python 1교시 60분

조건에 따라 실행 경로가 달라집니다



조건은 위에서부터 확인합니다. 실행할 분기를 예측한 뒤 TODO와 경계 검사를 비교하세요.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

배송비 규칙을 조건 순서로 옮기기

- 50,000원 이상이면 회원 여부와 무관하게 무료입니다.
- 그 미만에서 회원이면서 30,000원 이상이면 할인 배송비, 나머지는 기본 배송비입니다.
- 가장 넓은 우선 조건을 먼저 검사하고 경계 양쪽을 확인합니다.
- 함께 해석하기: 겹치는 조건이 있으면 먼저 만난 분기가 뒤의 규칙을 가릴 수 있습니다.

1. 금액 ≥ 50,000 → 0원 반환

2. 아니면 회원 & 금액 ≥ 30,000?

3. 위 조건 참 → 1,000원 반환

4. 나머지 → 3,000원 반환

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

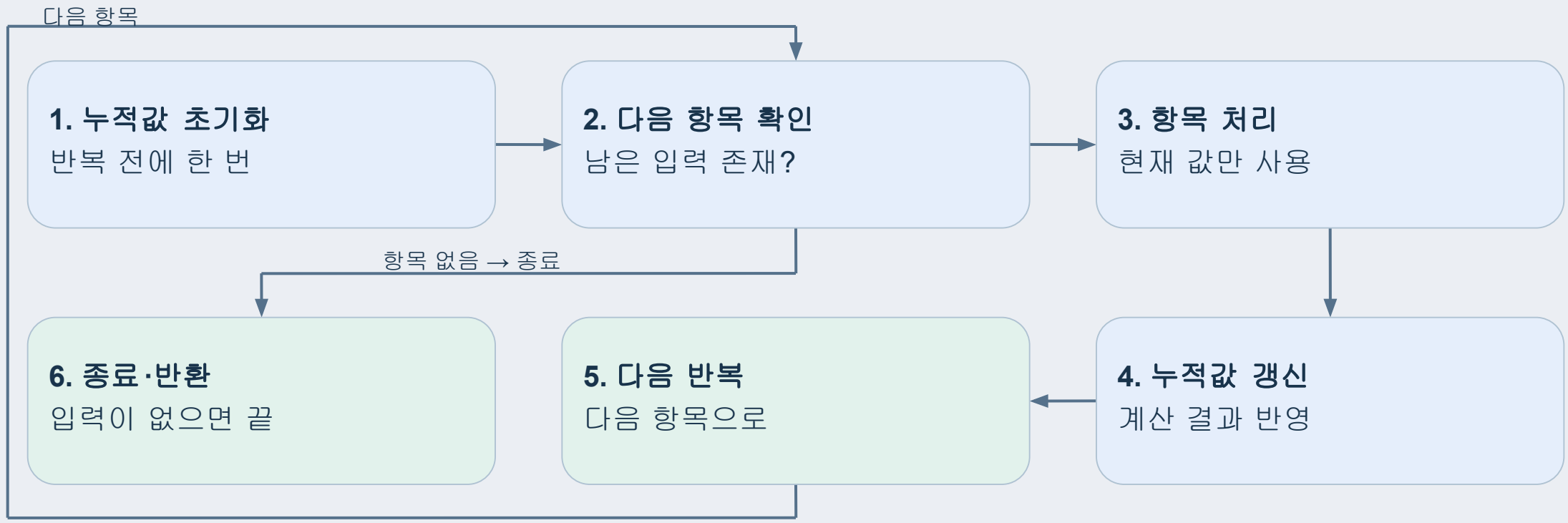
배송비 복습: 경계값으로 검증하기

- 01_python_basics.ipynb의 배송비 TODO를 수정하고 해당 셀을 실행한 결과를 확인합니다.
- 29,999·30,000·49,999·50,000원을 회원·비회원으로 비교합니다.
- 기대한 비용과 실제 반환값을 맞춘 뒤 검증 셀을 실행하고 노트북을 저장한 결과를 확인합니다.
- 함께 해석하기: 무료배송 경계·예외 지역·잘못된 입력 등 지정된 계약을 각각 확인합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

for: 자료를 하나씩 읽는 반복

반복의 시작·다음 항목·종료



2번에서 항목이 없으면 6번으로 종료합니다. 3–5는 항목마다 반복하며 초기화는 반복 밖에서 한 번 수행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

while·break·continue

- while은 조건이 참인 동안 반복하므로 종료 조건이 필요합니다.
- break는 반복 전체를 끝내고 continue는 이번 회차의 나머지를 건너뜁니다.
- 실행이 끝나지 않으면 조건이 바뀌는지 확인하고 커널 Interrupt나 Ctrl+C로 중단합니다.

```
count = 0
while count < 3:
    print(count)
    count += 1
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

함수: 입력과 결과의 약속

결과 복습 | 1일차 · Python 기본 문법 | Python 1교시 60분

함수 정의와 호출을 구분하기

정의만으로 본문이 실행되지는 않습니다. print는 표시, return은 호출자에게 값을 전달합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

지역 변수와 함수 호출 순서

- 함수 안에서 만든 지역 변수는 그 함수의 실행 범위에서 사용합니다.
- 같은 함수를 다른 인자로 여러 번 호출할 수 있습니다.
- 입력 리스트를 바꾸는 함수인지 새 결과만 반환하는 함수인지 계약을 확인합니다.
- 함께 해석하기: 지역 변수는 함수 범위의 이름입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Python 오류를 읽는 순서

결과 복습 | 1일차 · Python 기본 문법 | Python 1교시 60분

- NameError: 이름 정의·철자·앞 셀 실행을 확인합니다.
- TypeError: 값의 타입과 함수 인자를 확인합니다.
- AssertionError: 검증 기대값과 내 함수 반환값을 비교합니다.
- 수정 후에는 수정한 셀을 다시 실행해야 새 코드가 반영됩니다.
- 함께 해석하기: 오류 종류, 발생 셀·행, 실제 입력·타입, 실행 순서를 함께 확인합니다.

먼저 실습 | 02.Python/02_data_structures.ipynb

리스트·튜플·딕셔너리·센서 보고서 확인 후 다음 슬라이드로 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

자료구조는 조회 방식으로 선택합니다

- 리스트는 순서대로 모은 여러 값, 튜플은 바꾸지 않을 묶음에 적합합니다.
- 딕셔너리는 이름인 key로 값을 찾는 구조입니다.
- 자료의 이름만 외우기보다 순서·수정·조회 방법을 기준으로 선택합니다.
- 함께 해석하기: 순서로 읽을지, 한 기록의 값을 묶을지, 이름으로 찾을지에 따라 선택합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

리스트와 인덱스

- 인덱스는0부터 시작하며 마지막 유효 인덱스는 길이-1입니다.
- append는 끝에 값을 추가하고 len은 원소 개수를 반환합니다.
- 빈 리스트나 범위를 벗어난 조회는 별도 확인이 필요합니다.
- 함께 해석하기: 첫 원소는 0, 길이가 n이면 마지막 양의 인덱스는 n-1입니다.

```
values = [20, 30, 25]
print(values[0])
values.append(22)
print(len(values))
print(values[-1])
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

대입과 복사: 원본이 왜 바뀔까요?

- b=a는 같은 리스트를 가리키고 a.copy()는 새 리스트를 만듭니다.
- 중첩 리스트의 얕은 복사는 내부 객체까지 복사하지 않습니다.
- 실습: analyze_temperatures TODO를 완성합니다. 입력을 보존하고 원시 평균을 반환하며 표시할 때만 20.33으로 반올림합니다.
- 함께 해석하기: 두 이름이 같은 가변 리스트를 가리킵니다.

```
a = [10, 20]
b = a
c = a.copy()
b.append(30)
c[0] = 99
print(a, b, c)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

튜플과 언패킹

- 튜플은 순서 있는 값의 묶음이며 생성 뒤 원소를 직접 바꾸지 않습니다.
- 언패킹은 묶음의 값을 각각의 이름으로 받습니다.
- 이름과 점수처럼 의미가 다른 값들을 한 항목으로 다룰 때 유용합니다.
- 함께 해석하기: 계약에 맞지 않아 오류가 납니다.

```
record = ("A", 25.0)
name, value = record
print(name)
print(value)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

key 함수로 최고 점수의 원본 기록 고르기

- max의 key 함수는 각 기록에서 비교할 점수를 반환합니다.
- max는 점수가 아닌 최고 점수의 원본 튜플 하나를 반환합니다.
- 정렬된 전체 리스트가 필요하면 sorted를 사용합니다.
- 함께 해석하기: 최고 점수의 원본 튜플 ('준호',95)입니다.

```
records = [("민지", 88), ("준호", 95)]
def score_of(row):
    return row[1]
best = max(records, key=score_of)
print(best)  # ('준호', 95)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

딕셔너리: key로 값 찾기

- key는 값을 찾는 이름이며 같은 key는 하나의 현재 값에 연결됩니다.
- 없는 key를 []로 조회하면 KeyError가 납니다. get은 기본값을 지정할 수 있습니다.
- items는 key와 값을 함께 순회할 때 사용합니다.
- 함께 해석하기: 대괄호 조회는 KeyError가 날 수 있습니다.

```
sensor = {"A": 25, "B": 30}
print(sensor["A"])
print(sensor.get("C", None))
for name, value in sensor.items():
    print(name, value)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

설비별 그룹을 먼저 만들기

센서 기록에서 보고서까지

원본 → 그룹 → 요약을 각각 확인합니다. 그룹 오류는 통계 함수에서 숨기지 않습니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

센서 그룹 구현 확인

- 02_data_structures.ipynb에서 group_sensor_readings TODO 구현 결과를 확인합니다.
- 설비별 원소 수와 실제 값 목록을 출력해 원본 기록과 비교합니다.
- 그룹 검증을 통과한 뒤 summarize_sensor_groups로 넘어갑니다.
- 함께 해석하기: 그 설비의 목록을 준비한 뒤 값을 추가합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

평균·개수·경보를 따로 읽기

- 평균은 합계를 개수로 나눈 값이며 순간 최고값이나 변동을 모두 보여주지 않습니다.
- [20,20,20]과 [10,20,30]은 평균이 같지만 변동은 다릅니다.
- count·mean·alarm의 정의와 단위를 확인하고 누락값이 있는 그룹을 무조건 비교하지 않습니다.
- 이 실습의 alarm은 평균이 아니라 max(values) ≥ 30.0일 때 참입니다.
- 함께 해석하기: 평균은 합계/개수이고 경보는 과제가 정한 개별 값의 임계 조건입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

센서 보고서 결과와 완료 기준

- summarize_sensor_groups TODO에서 count·mean·alarm을 계산합니다.
- 정상·빈 입력·경계 입력 검증을 확인합니다.
- 완료된 보고서를 Data/outputs/02.Python/sensor_report.json에 저장하고 다시 읽습니다.
- 함께 해석하기: 현재 과제는 원본 보존도 계약입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

JSON: 프로그램 결과를 파일로 남기기

- JSON은 딕셔너리·리스트 같은 구조를 다른 프로그램에서도 읽을 수 있는 텍스트로 표현합니다.
- 파일 저장은 커널 변수와 달리 컨테이너 재생성 뒤에도 재사용할 수 있습니다.
- 문자열·숫자·불리언의 타입을 유지하고 다시 읽어 값이 같은지 확인합니다.
- 함께 해석하기: 다시 읽어 구조·값이 메모리 결과와 같은지 확인합니다.

```
import json
from pathlib import Path
base = Path("/workspace/Data/outputs")
p = base / "demo.json"
base.mkdir(
    parents=True, exist_ok=True)
p.write_text(json.dumps({"mean": 25}))
print(json.loads(p.read_text()))
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Python 마무리: 구현과 판단 두 결과

결과 복습 | 1일차 · Python 자료구조와 센서 보고서 | Python 2교시 60분

- 구현: 배송비 경계 검사와 센서 그룹·통계·JSON 저장을 확인합니다.
- 판단: 평균만으로 알 수 없는 것 한 가지와 필요한 추가 정보를 적습니다.
- 노트북과 결과 파일을 저장하고 C++에서 같은 계산을 다시 표현할 준비를 합니다.
- 함께 해석하기: 내 함수가 어떤 입력 계약을 처리하고 어떤 경계를 검증했는지, 저장된 결과가 무엇을 뜻하는지 설명합니다.

먼저 실습 | 03.C++/01_cpp_basics.ipynb

기초 문법·영화표 계산기 확인 후 다음 슬라이드로 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

C++를 배우는 이유와 오늘의 범위

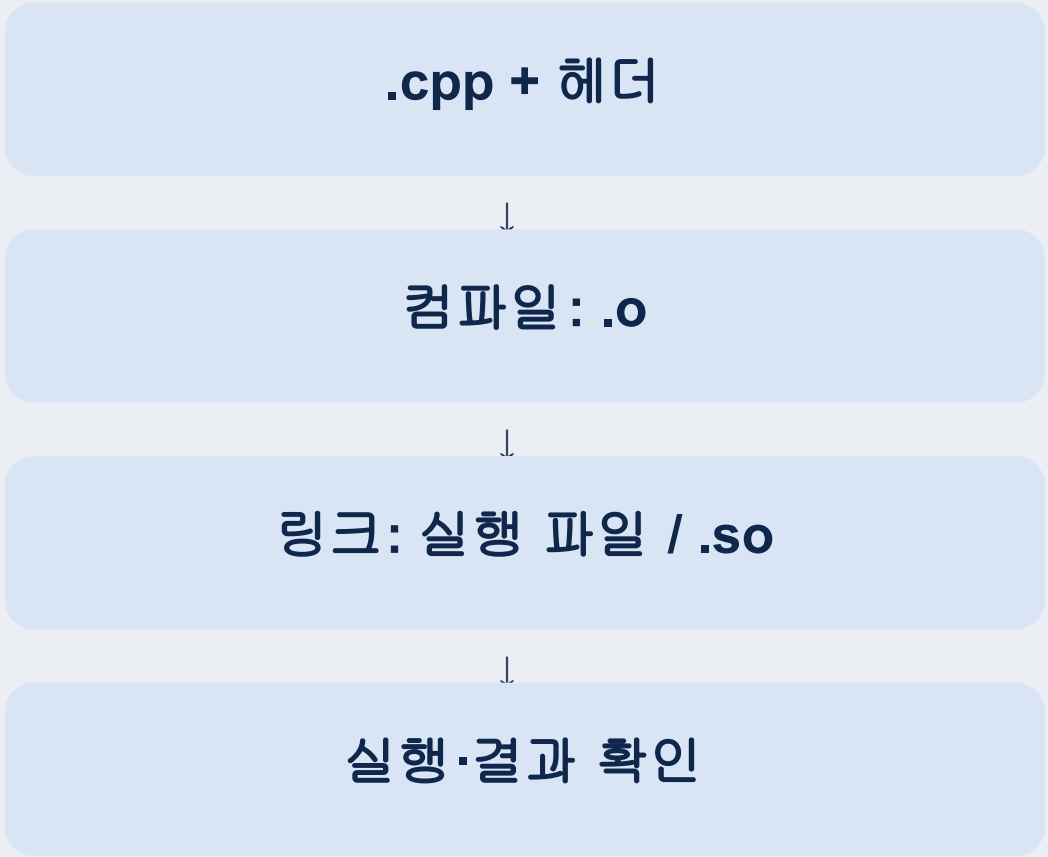
- C++는 자료형·메모리·빌드 과정을 명시하며 로봇·영상 프로그램에서 자주 만납니다.
- Python 경험을 같은 입력·계산·출력 구조에 연결합니다.
- 오늘은 제공된 뼈대의 TODO를 수정하고 마지막에 .so를 Python에서 호출합니다.
- 함께 해석하기: 성능뿐 아니라 기존 라이브러리 재사용·타입·메모리와 호출 경계의 이해가 목적입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

소스·헤더·목적 파일·실행 파일

ALL AI

- 소스 .cpp는 구현, 헤더 .h/.hpp는 공개 선언을 담습니다.
- 컴파일은 소스를 기계어 조각으로, 링크는 필요한 조각을 실행 파일로 연결합니다.
- 소스를 수정하면 저장한 뒤 다시 빌드해야 실행 결과가 바뀝니다.
- 함께 해석하기: 선언·구현 소스가 컴파일되고 필요한 목적 파일이 링크되어야 실행 파일이 생깁니다.



본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Jupyter에서 C++를 실행하는 실제 경로

노트북에서 C++ 실행까지

컴파일 성공과 정답 통과는 다릅니다. 오류가 난 단계로 돌아가 수정·저장부터 다시 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

main·중괄호·세미콜론

- main은 실행 파일의 시작 함수입니다.
- 중괄호는 실행 묶음을 만들고 세미콜론은 많은 문장을 끝냅니다.
- #include는 필요한 선언을 가져오며 std::cout은 출력에 사용합니다.
- 함께 해석하기: 주로 중괄호가 범위를 정하고 세미콜론이 문장을 끝냅니다.

```
#include <iostream>
int main() {
    int value = 25;
    std::cout << value << "\n";
    return 0;
}
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

타입·초기화·const

- int는 정수, double은 실수, bool은 참·거짓을 나타냅니다.
- 변수는 사용 전에 초기화하고 가능한 값 범위를 생각합니다.
- const는 그 이름을 통한 값 변경을 제한하여 의도를 드러냅니다.
- 함께 해석하기: 그렇게 가정하면 안 됩니다. 선언할 때 의미 있는 값으로 초기화하고 const는 바뀌지 않아야 하는 값을 표현합니다.

```
int count = 3;
double temperature = 25.5;
bool alarm = temperature >= 30;
const double limit = 30.0;
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

정수 나눗셈과 타입 변환

- 정수끼리 나누면 정수 결과가 됩니다. 평균처럼 실수가 필요하면 계산 전에 변환합니다.
- 계산이 끝난 뒤 double에 저장해도 이미 버린 소수는 되살아나지 않습니다.
- 범위·타입을 알고 변환해야 의도한 결과를 얻습니다.
- 함께 해석하기: 정수 나눗셈은 이 예제에서 2입니다.

```
int sum = 5;
int count = 2;
double a = sum / count;
double b =
    static_cast<double>(sum) / count;
// a: 2, b: 2.5
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

C++ 조건문과 반복문

결과 복습 | 1일차 · C++ 기본 문법 | C++ 1교시 60분

- if·else의 조건은 Python과 같은 판단 역할을 하지만 괄호·중괄호 문법이 다릅니다.
- for는 초기값·계속 조건·갱신을 명시할 수 있습니다.
- 배열 인덱스는 길이보다 작은 동안만 사용합니다.
- 함께 해석하기: 규칙은 같아도 타입·연산·문법이 달라질 수 있습니다.

```
int total = 0;
for (int i = 1; i <= 3; ++i) {
    total += i;
}
if (total == 6) {
    // 기대한 합계
}
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

영화표 계산기 구현 확인

- 01_cpp_basics.ipynb의 지정 TODO에서 가격·조건 계산을 완성합니다.
- writefile 셀을 실행하고 build→run→학습자 검증 순서로 확인합니다.
- 경계 입력의 기대값과 실제값을 비교하고 노트북·소스를 저장한 결과를 확인합니다.
- 함께 해석하기: 할인액은 3,600원, 결제액은 32,400원입니다.

먼저 실습 | 03.C++/02_functions_classes.ipynb
함수·전달 방식·장바구니 확인 후 다음 슬라이드로 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

함수 선언과 정의

- 선언은 이름·매개변수·반환 타입의 약속이며 정의는 실제 처리 내용입니다.
- 호출할 때 인자의 개수·순서·타입이 약속과 맞아야 합니다.
- 반환값과 원본 상태 변경을 구분해 함수를 읽습니다.
- 함께 해석하기: 호출 형식은 알려져 컴파일을 지나도 구현을 연결하지 못하면 링크에서 실패할 수 있습니다.

```
double add(double a, double b);

double add(double a, double b) {
    return a + b;
}
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

값 전달: 함수 안의 복사본

- 값으로 전달하면 함수 매개변수는 전달값의 복사본입니다.
- 함수 안에서 매개변수를 바꿔도 원래 변수는 그대로입니다.
- 큰 객체의 복사 비용과 원본 수정 의도를 따져 전달 방법을 선택합니다.
- 함께 해석하기: 기본 값 전달 예제는 복사본을 바꿉니다.

```
void increment(int x) {
    x += 1;
}
int a = 2;
increment(a);
// a는 2
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

참조 전달: 원본을 다른 이름으로 사용

- int&는 원본 변수에 대한 참조를 받습니다.
- 참조 매개변수를 수정하면 호출한 쪽의 원본도 바뀝니다.
- 변경이 함수의 의도인지 인터페이스와 검증에서 드러내야 합니다.
- 함께 해석하기: 원본을 다른 이름으로 사용하는 방식입니다.

```
void increment(int& x) {
    x += 1;
}
int a = 2;
increment(a);
// a는 3
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

const 참조: 복사 없이 읽기

- const T&는 큰 객체를 복사하지 않고 읽기 위해 자주 사용합니다.
- 함수는 이 참조를 통해 객체를 수정하지 못합니다.
- 원본 객체의 수명은 호출 동안 유효해야 합니다.
- 함께 해석하기: 복사 비용 없이 원본을 읽되 그 참조를 통해 변경하지 않겠다는 약속입니다.

```
double total(
    const std::vector<double>& values
);
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

구조체: 관련 값을 하나로 묶기

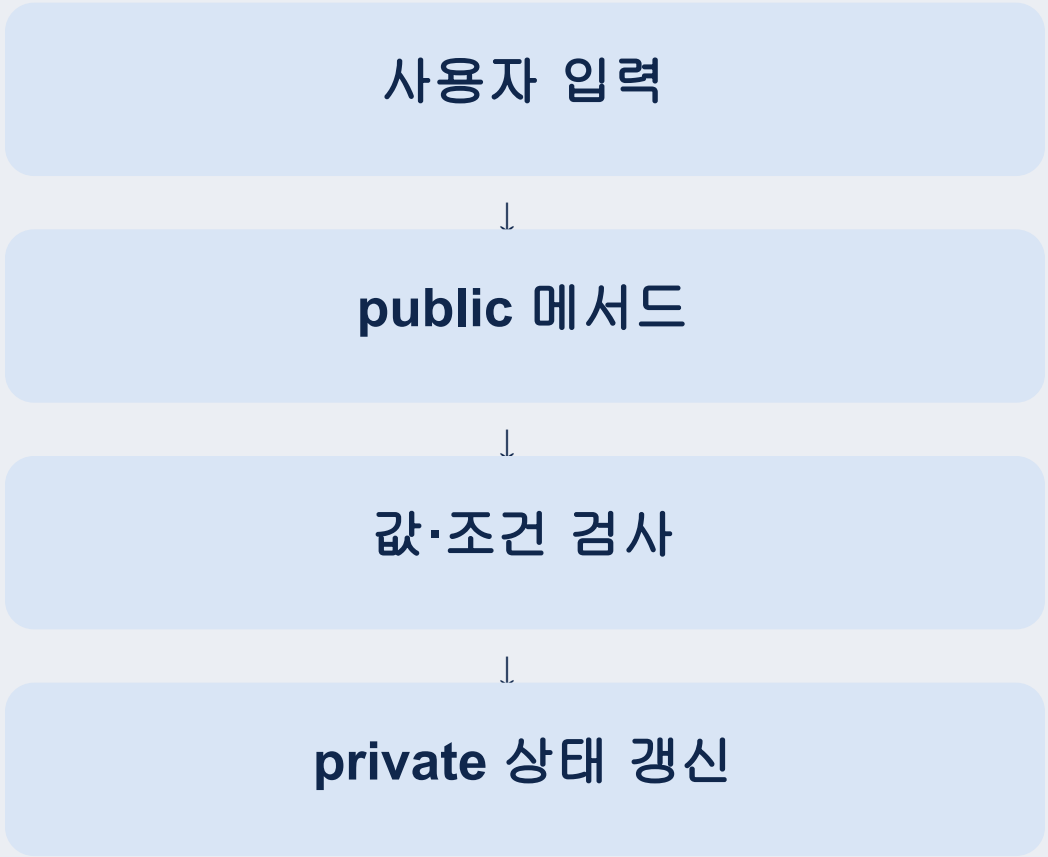
- struct는 상품 이름·가격처럼 관련 필드를 하나의 타입으로 묶습니다.
- 필드 이름을 사용하면 리스트의 몇 번째 값인지 외우는 부담이 줄어듭니다.
- 객체 하나와 객체 여러 개를 담는 컨테이너를 구분합니다.
- 함께 해석하기: 같은 계산의 관련 결과를 이름 있는 필드로 전달해 순서 혼동을 줄입니다.

```
struct Product {
    std::string name;
    double price;
};
Product item{"sensor", 100.0};
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

클래스: 유효한 상태를 지키기

- 클래스는 상태와 그 상태를 다루는 함수를 함께 묶습니다.
- public은 외부 인터페이스, private은 내부 상태 관리에 사용합니다.
- 음수 수량처럼 허용하지 않을 상태는 입력 경계에서 검사합니다.
- 함께 해석하기: 멤버 함수를 통해 유효성을 검사하고 실패할 때 상태를 보존하도록 구현해야 합니다.

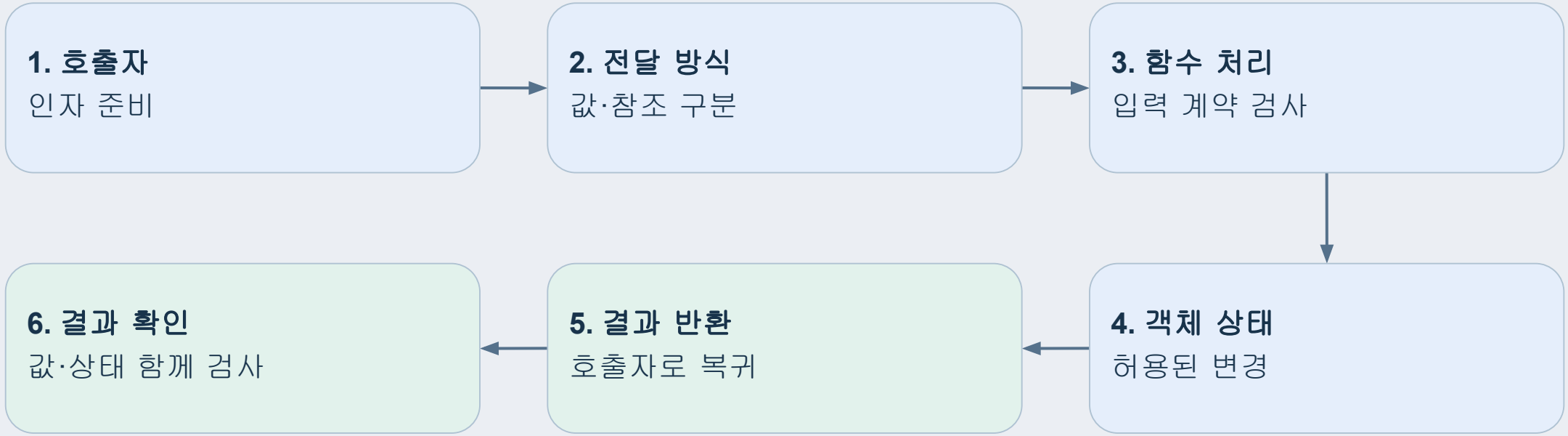


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

장바구니 결과: 상태 변화 확인

결과 복습 | 1일차 · C++ 함수·상태 | C++ 2교시 60분

함수와 객체를 따라 값 추적하기



값 전달은 복사, 참조 전달은 원본과 연결됩니다. 상태를 바꾸는 함수인지 먼저 확인합니다.

먼저 실습 | 03.C++/03_memory_stl_cmake.ipynb

센서 통계 TODO·빌드·CTest 확인 후 다음 슬라이드로 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

vector: 길이가 변하는 같은 타입의 목록

- std::vector는 같은 타입 원소를 순서대로 저장합니다.
- size로 길이를 확인하고 push_back으로 끝에 추가합니다.
- 크기 변경으로 내부 메모리가 이동하면 기존 원소의 주소가 무효가 될 수 있습니다.
- 함께 해석하기: 재할당이 일어나면 이전 포인터·참조·iterator가 무효가 될 수 있습니다.

```
std::vector<double> values{20, 30};
values.push_back(25);
for (double value : values) {
    // value를 읽어 계산
}
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

주소·포인터·역참조

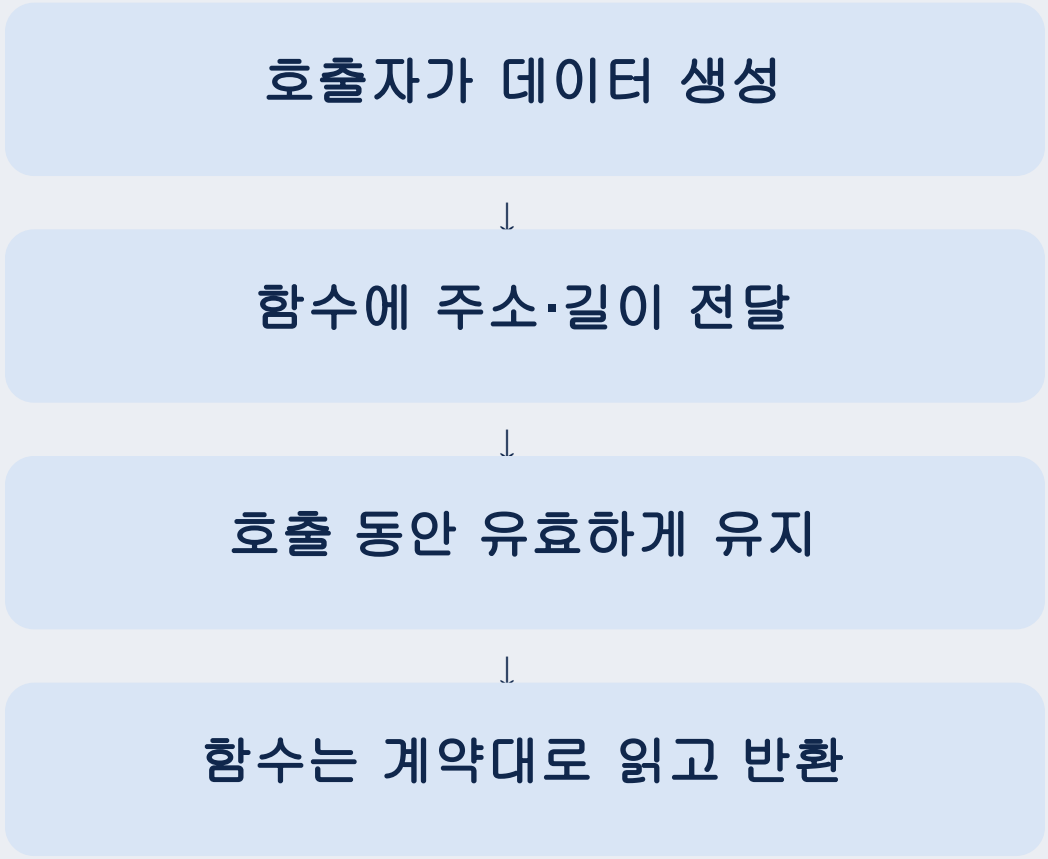
- 주소는 값이 있는 메모리 위치, 포인터는 그 주소를 저장하는 값입니다.
- &는 문맥에 따라 주소를 얻거나 참조 타입을 선언할 때 쓰입니다.
- *로 포인터가 가리키는 값을 읽거나 수정하려면 유효한 주소여야 합니다.
- 함께 해석하기: 포인터는 주소를 보관하고 역참조는 그 주소의 대상 값에 접근합니다.

```
int value = 10;
int* p = &value;
*p = 20;
// value는 20
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

수명과 소유권

- 지역 객체는 범위를 벗어나면 수명이 끝납니다.
- 수명이 끝난 객체의 주소를 사용하면 정상 동작을 보장할 수 없습니다.
- 주소를 빌려 쓰는 함수와 메모리를 관리하는 소유자의 책임을 구분합니다.
- 함께 해석하기: 지역 객체의 수명이 끝나면 그 주소를 통한 사용은 유효하지 않습니다.



본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

포인터에는 배열 길이가 들어 있지 않습니다

- 포인터만으로 몇 개의 원소가 유효한지 알 수 없습니다.
- 주소와 길이를 함께 전달하고 null·빈 입력·범위 밖 접근을 검사합니다.
- const double*는 이 포인터를 통한 원소 변경을 제한합니다.
- 함께 해석하기: 포인터 자체에는 길이가 없습니다.

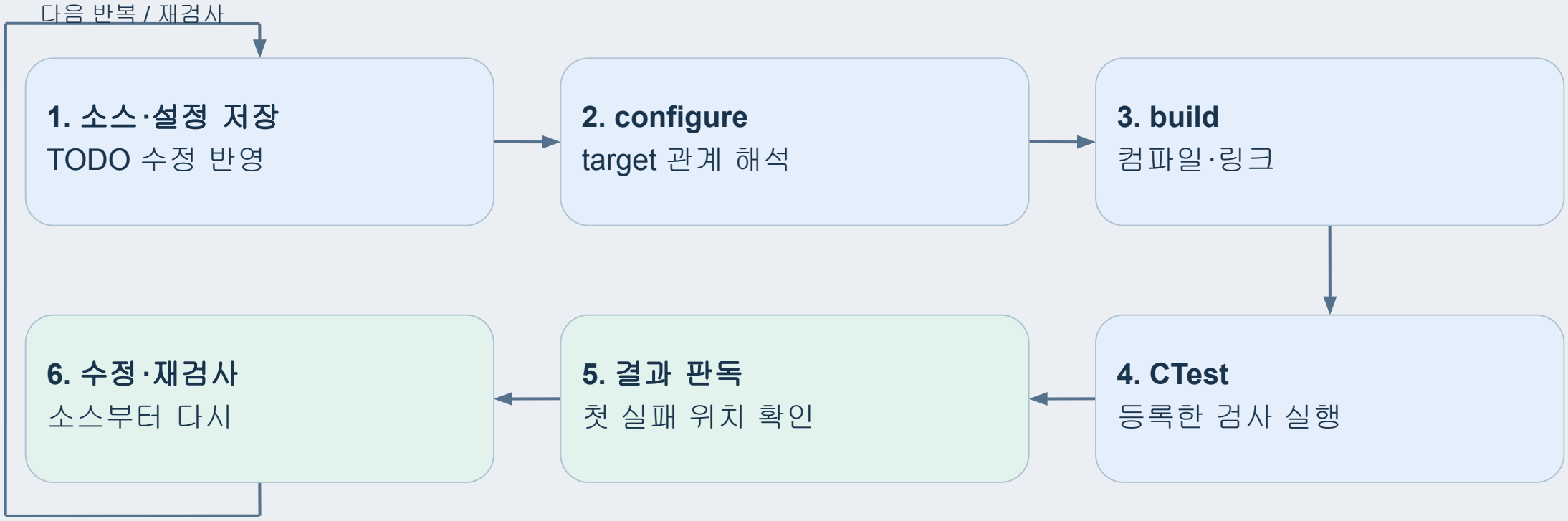
```
const double* values;
std::size_t count;
// 주소와 count가 같은 배열을 설명해야 함
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

CMake: 빌드 대상을 연결하는 도구

결과 복습 | 1일차 · 메모리·STL·CMake | C++ 3교시 60분

CMake로 빌드하고 검사하기



빌드 오류와 검사 실패를 구분합니다. 산출물은 Data/build에, 수정 원본은 교재 폴더에 둡니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

configure·build·test 명령 읽기

- -S는 소스 폴더, -B는 빌드 결과 폴더입니다.
- Data/build 아래에 결과를 분리하면 교재 소스와 생성물을 구분하기 쉽습니다.
- 실제 실습은 노트북이 제공한 경로·명령을 그대로 확인합니다.
- 함께 해석하기: 설정 생성, 산출물 빌드, 등록된 검사 실행입니다.

```
cmake -S 소스폴더 -B 빌드폴더
cmake --build 빌드폴더
ctest --test-dir 빌드폴더 \
  --output-on-failure
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

센서 통계 C++ 결과

결과 복습 | 1일차 · 메모리·STL·CMake | C++ 3교시 60분

- 수업 필수: vector·const 참조·비소유 포인터와 CMake·센서 TODO입니다.
- 원본 입력 보존→빌드→CTest→결과 저장을 확인합니다.
- array·tuple·map 세부 비교와 직접 동적 할당은 복습·강사 시연입니다.
- 함께 해석하기: count=3, mean=25, alarm=true입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

컴파일·링크·실행 오류 구분

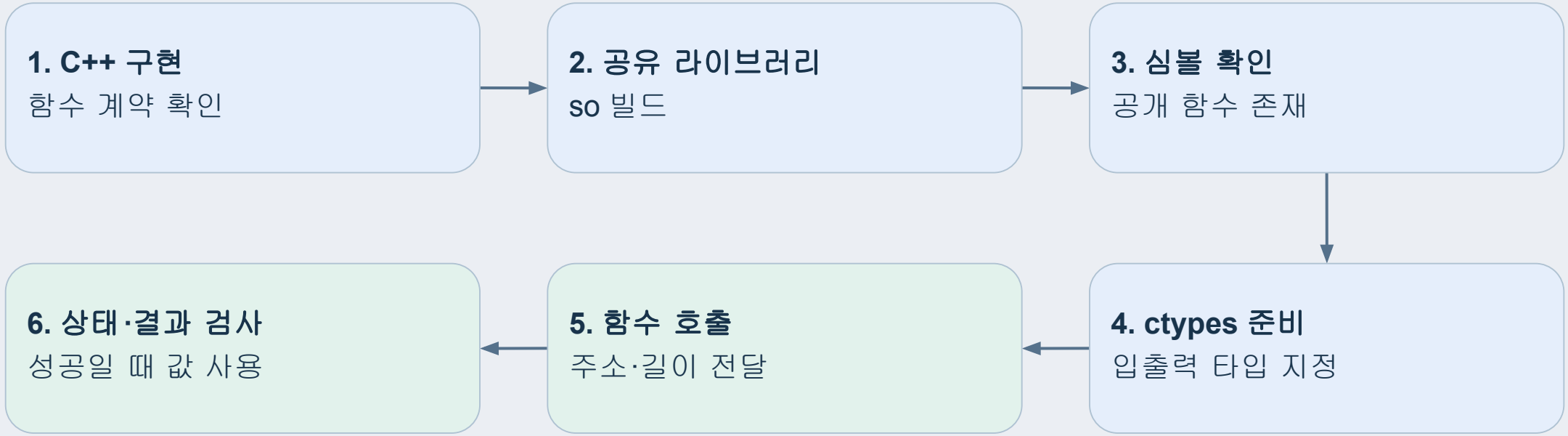
- 컴파일 오류: 문법·타입·선언을 확인합니다.
- 링크 오류 undefined reference: 정의 소스·라이브러리 연결을 확인합니다.
- 실행 오류: 입력·주소·길이·수명·검증 기대값을 확인합니다.
- Python의 CalledProcessError 위에 실제 C++ 오류가 출력될 수 있습니다.
- 함께 해석하기: 문법·타입의 컴파일 오류, 구현 연결의 링크 오류, 실행 중 입력·계약 오류를 구분합니다.

먼저 실습 | 03.C++/04_shared_library_python.ipynb
.so 빌드·ctypes 검증·결과 저장 확인 후 다음 슬라이드로 진행합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

공유 라이브러리 .so는 재사용할 코드 묶음

C++ 함수를 Python에서 호출하기



재빌드한 so는 새 프로세스에서 다시 로드해 확인합니다. 반환 상태와 계산 결과를 구분하세요.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

C ABI: 언어 사이의 호출 약속

- ABI는 함수 호출과 데이터 표현 등 바이너리 수준의 약속입니다.
- extern "C"는 C++ 함수 이름의 C 언어 연결 방식을 지정합니다.
- 이것만으로 모든 C++ 객체가 Python과 자동 호환되는 것은 아닙니다.
- 함께 해석하기: 호출 이름과 형식을 연결하기 위한 경계이며 타입·길이·소유권·오류 처리도 따로 약속해야 합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

sensor_mean 계약을 읽기

- values는 읽을 double 배열 주소, n은 원소 수입니다.
- out_mean은 평균을 쓸 공간의 주소입니다.
- 함수 반환 int는 평균이 아니라 성공·실패 상태입니다.
- 함께 해석하기: 실패 여부와 계산값은 다른 정보입니다.

```
int sensor_mean(
    const double* values,
    size_t n,
    double* out_mean
);
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

ctypes에서 타입을 정확히 지정하기

- argtypes는 C 함수 인자의 타입과 순서를 지정합니다.
- restype은 함수가 반환하는 값의 타입입니다.
- double 포인터·size_t·double 포인터·int 계약을 Python에서도 똑같이 맞춥니다.
- 코드는 앞서 library를 로드한 뒤 실행하는 발췌입니다.

```
import ctypes as ct
fn = library.sensor_mean
fn.argtypes = [
    ct.POINTER(ct.c_double),
    ct.c_size_t,
    ct.POINTER(ct.c_double),
]
fn.restype = ct.c_int
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

배열·출력 공간·상태를 분리하기

- 호출자는 C가 읽을 배열과 결과를 쓸 double 공간을 만듭니다.
- 호출이 끝날 때까지 배열을 유지하고 상태 코드를 먼저 검사합니다.
- 성공했을 때만 출력 공간의 value를 평균으로 사용합니다.
- 함께 해석하기: 호출 동안 유효한 메모리를 유지해야 합니다.

```
array = (ct.c_double * 3)(
    20, 30, 25)
output = ct.c_double()
status = fn(
    array, 3, ct.byref(output))
if status == 0:
    print(output.value)
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

공유 라이브러리 빌드·호출 실습

결과 복습 | 1일차 · C++ 공유 라이브러리와 Python | C++ 4교시 60분

- 04_shared_library_python.ipynb의 sensor.cpp TODO를 확인합니다.
- 필수: .so 빌드→ctypes 호출→유한성·실패 시 출력 보존 검사.
- NaN·Inf·overflow 상세 원리와 심볼 해석은 복습·강사 시연입니다.
- ABI 검증과 Python/C++ 평균 비교 후 결과 JSON을 저장합니다.
- 저장 경로: Data/outputs/03.C++/sensor_mean.json

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

같은 입력으로 Python과 C++ 비교하기

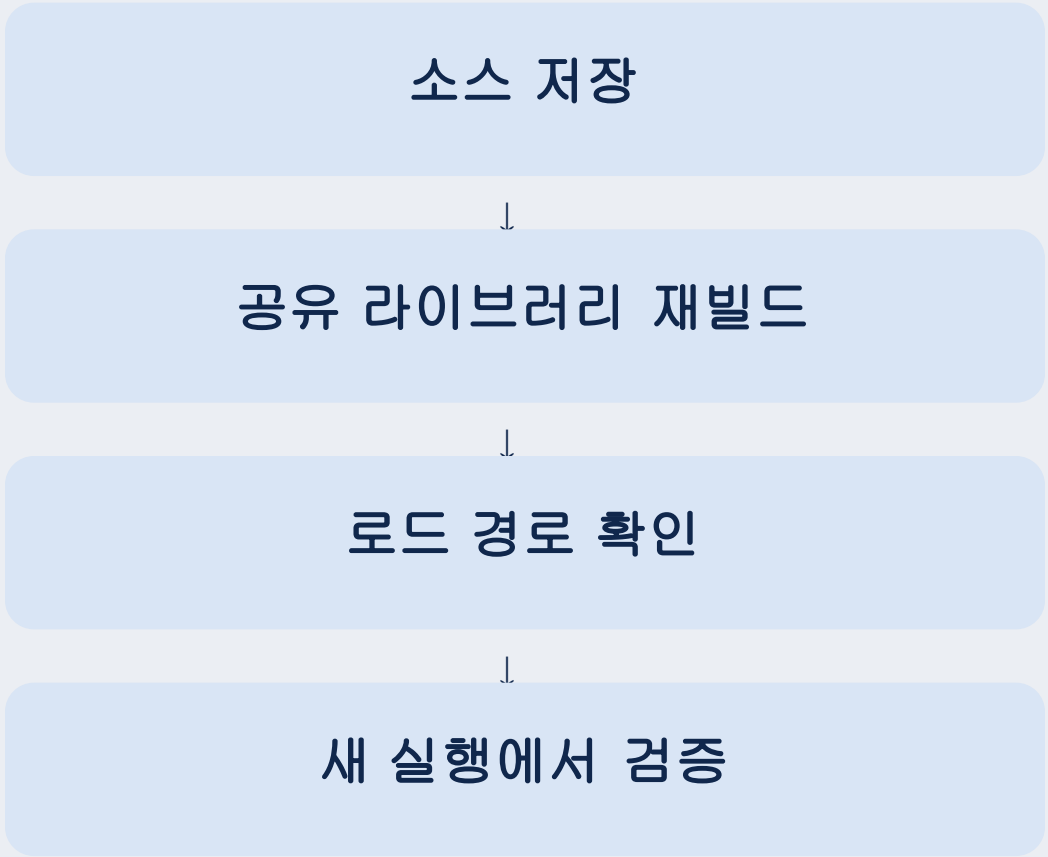
- [20,30,25]의 평균은 두 경로에서25입니다.
- 부동소수점 비교는 목적에 맞는 허용오차를 사용합니다.
- 입력·Python 결과·C++ 결과·오류 상태를 함께 기록하면 차이를 재현할 수 있습니다.
- 앞 실습 wrapper로 actual을 얻은 뒤 비교하는 코드 발췌입니다.

```
values = [20.0, 30.0, 25.0]
expected = sum(values) / len(values)
# actual: C++ wrapper 반환값
assert abs(actual - expected) < 1e-12
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

소스를 바꾼 뒤 결과가 그대로라면

- 수정한 소스를 저장하고 .so를 다시 빌드했는지 확인합니다.
- Python 프로세스가 이전 라이브러리를 이미 로드했을 수 있습니다.
- 노트북 안내에 따라 새 프로세스 또는 커널에서 새 라이브러리를 로드합니다.
- 함께 해석하기: 실제 로드 경로와 프로세스가 이미 읽은 라이브러리를 확인합니다.



본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

C++로 분리할 이유를 설명하기

- 기존 계산 부품 재사용, 필요한 성능, 하드웨어 연동 요구가 분리 이유가 될 수 있습니다.
- Python↔C++ 호출과 데이터 변환에도 비용이 듭니다.
- 구현 결과 일치·호출비용 실측·유지보수 부담을 함께 비교합니다.
- 함께 해석하기: 호출 비용과 전체 병목을 측정해야 합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

오늘의 결과 파일 점검

결과 복습 | 1일차 · 1일차 정리 | 저장·복구·다음 일차 연결

- Python 센서 보고서와 수정한 노트북·소스를 확인합니다.
- C++ 테스트 결과와 Data/outputs/03.C++/sensor_mean.json을 확인합니다.
- 파일 브라우저에서 다시 열어 내용·입력·예상 결과가 일치하는지 확인합니다.
- 함께 해석하기: 수정한 노트북·학생 소스와 지정한 Data 결과를 저장합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

종료·재접속·재생성 순서

1일차 · 1일차 정리 | 저장·복구·다음 일차 연결

- 파일 저장→실습 프로세스 종료→컨테이너 종료 순서로 진행합니다.
- --rm으로 삭제된 컨테이너는 docker start가 아니라 기존 생성 명령으로 다시 만듭니다.
- 재접속 후 파일 보존을 확인하고 환경 셀부터 실행합니다.
- 함께 해석하기: 연결된 호스트 개인 폴더는 보존됩니다.

```
# Jetson 호스트: 본인 컨테이너만
docker stop physical_ai_remote01
# 이후 같은 run 스크립트로 재생성
# 파일은 남고 커널 변수는 초기화
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

이해 확인: 세 질문

- 원격 브라우저에서 실행한 Python은 어느 컴퓨터에서 계산하나요?
- 컨테이너를 삭제했는데 남는 파일과 사라지는 상태는 무엇인가요?
- C 함수의 상태 코드와 평균 출력값은 어떻게 구분하나요?
- 함께 해석하기: 어디에서 실행했는지, 무엇을 수정했는지, 어떤 입력에서 어떤 검증·출력이 나왔는지를 연결합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

2일차 연결: 데이터에서 모델로

1일차 · 1일차 정리 | 저장·복구·다음 일차 연결

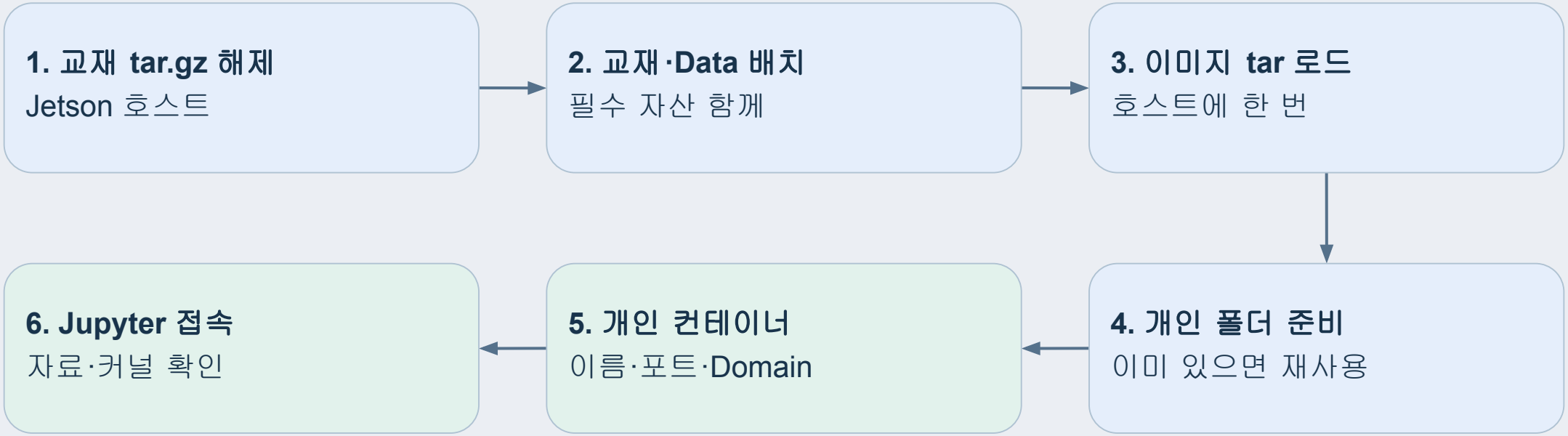
- 오늘 만든 리스트·딕셔너리는 모델 입력과 API 메시지를 읽는 기초가 됩니다.
- 함수의 입력·반환·오류 계약은 LLM·VLM 응답 처리에도 적용됩니다.
- 다음 시간은 작은 모델의 훈련과 사전학습 모델의 추론을 구분하며 시작합니다.
- 함께 해석하기: 모델도 정해진 타입·shape·범위의 입력을 사용합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

배포 파일 하나에 교재·Data·이미지 3개

ALL AI

교재 수신에서 개인 실습까지



바깥 압축 해제와 내부 이미지 docker load는 다른 단계입니다. 공용 이미지는 한 번 로드하고 개인 실행 환경을 만듭니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

배포 파일 구성과 수강생 준비 범위

1일차 · 부록 / 자습 | 접속·배포본 사용 참고

- 2~5쪽을 완료했다면 준비 작업을 반복하지 않습니다.
- Ollama 모델·5일차 필수 캐시는 바깥 압축에 함께 배치됩니다.
- 파일 무결성·이미지 ID 검사는 강사가 배포 전에 수행합니다.
- 수강생은 별도 파일 검사·모델 다운로드·이미지 빌드를 하지 않습니다.

```
# 최초 준비 · 2~5쪽 순서
# 1. Jetson에서 바깥 압축 해제
# 2. 내부 이미지 tar를 한 번 load
# 3. 개인 폴더·컨테이너 준비
# 4. Jupyter 접속·파일 저장

# 이미 준비했다면 접속부터 진행
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

개인 자료 복사: 필수 캐시 포함

- Jetson 호스트에서 배정된 개인 폴더명으로 실행합니다.
- 개인 결과와 공용 Ollama 저장소는 제외합니다.
- 참고 가중치·5일차 캐시·LIBERO 자산은 포함합니다.
- 이미 존재하는 목적지는 덮어쓰지 않습니다.

```
cd /home/nvidia/NEPES
bash 01.Setup/02.Docker/\
07_copy_workspace.sh \
  /home/nvidia/NEPES \
  /home/nvidia/NEPES_remote01
cd /home/nvidia/NEPES_remote01
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

SSH와 원격 호스트 셸

- SSH는 원격 장비의 셸을 사용하는 방법입니다. 브라우저의 Jupyter 접속과 별개입니다.
- 개인 PC에서 ssh 계정@Jetson-IP로 접속한 뒤 호스트 명령을 실행합니다.
- 계정·IP·네트워크·SSH 서비스는 강사가 준비한 값을 사용합니다.
- 함께 해석하기: SSH 안에서는 Jetson 호스트의 현재 경로가 바뀝니다.

```
# 개인 PC 터미널: 값은 현장 배정
ssh 계정@Jetson-IP
# 접속 후 Jetson 호스트 셸
hostname
pwd
docker ps
```


본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

sudo·권한·패키지 설치의 역할

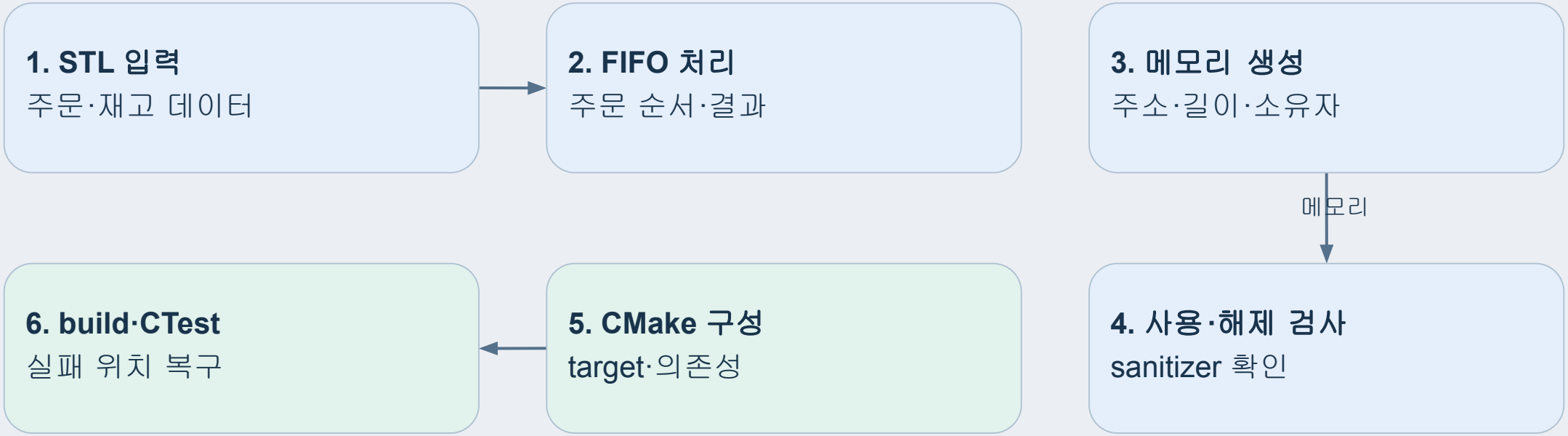
- sudo는 관리자 권한으로 명령을 실행합니다. 권한 오류의 원인을 먼저 확인합니다.
- Docker 그룹·NVIDIA 런타임·패키지 설치는 로컬 담당자의 호스트 준비 항목입니다.
- 수강생은 준비된 컨테이너를 사용하며 sudo나 전체 재설치를 기본 복구 절차로 삼지 않습니다.
- 함께 해석하기: 실행 위치·파일 소유권·마운트·장치 권한을 먼저 확인합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이 무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

선택 학습: 클래스·큐·상세 메모리

1일차 · 부록 / 자습 | 접속·배포본 사용 참고

C++ 선택 자습의 세 경로



1–2는 STL, 3–4는 메모리, 5–6은 CMake의 독립 자습 경로입니다. 공통적으로 입력·처리·검증 근거를 남깁니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

개별 실습을 위한 초기 설정

1일차 v2 · 부록 / 안내

ALL AI

- 이 부록은 Orin Nano Developer Kit에서 개별 실행 환경을 구성하기 위한 참고 자료입니다.
- Windows + balenaEtcher + JetPack 6.2 SD 카드 이미지를 기준으로 설명합니다.
- 실제 장비에서 추가 검증이 필요합니다. 펌웨어·패키지 버전에 따라 결과가 달라질 수 있습니다.
- 오류가 나면 실행 명령·오류 메시지·JetPack/L4T 버전을 기록하고 다음 장의 공식 문서와 포럼에서 검색하세요.
- 문서 확인과 실제 실행 검증은 다릅니다. 이 부록의 설치·GPU 동작은 실장비 검증 전입니다.

읽는 순서: 공통 초기 설정 → 필요한 설치 절 선택 → 해당 절의 완료 확인

↗ JetPack 6.2 설치 안내

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

오류가 발생하면 어디를 검색할까?

ALL AI

- SD 기록·검증 실패 → balenaEtcher / balena 포럼
- 부팅·펌웨어·GPU·L4T → NVIDIA Jetson 포럼 / NGC 이미지 설명
- Docker 설치·권한 → Docker 공식 문서
- ROS 2 설치·통신 → Humble 공식 문서 / Robotics Stack Exchange
- LLM 설치·추론 → Ollama Linux / Troubleshooting
- 검색 예: Orin Nano JetPack 6.2 + 오류 핵심 문장. 검색 결과의 버전도 확인합니다.

아래 출처 제목은 클릭 가능합니다. 마지막 보조 문서에는 전체 URL을 제공합니다.

↗ balena 포럼 ↗ NVIDIA Jetson 포럼 ↗ Docker Ubuntu 설치 ↗ Robotics Stack Exchange ↗ Ollama 문제 해결

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

필요한 경로만 독립적으로 실행합니다

ALL AI

- 공통: A. Windows 플래싱 → B. Jetson 첫 부팅·초기 설정
- 컨테이너 사용: B 완료 → C. Docker 설치 → D. L4T 이미지 받기·실행
- ROS 2 직접 설치: B 완료 → E. 호스트 ROS 2 (Docker 불필요)
- LLM 직접 설치: B 완료 → F. 호스트 Ollama·Python (Docker·ROS 불필요)
- Windows PC는 이미지 기록용입니다. 이후 호스트 명령은 Jetson Ubuntu의 Bash에서 실행합니다.
- 각 절은 새 환경용 예제입니다. 이미 설치된 경우 버전·서비스를 확인한 뒤 필요한 단계만 수행합니다.

이 부록의 기본 환경 구축과 기존 강의용 v1.0 이미지 사용은 별도 경로입니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

준비물과 버전 기준

1일차 v2 · 부록 / A · Windows

ALL AI

- 장비: Jetson Orin Nano Developer Kit, 전원, DisplayPort 모니터, USB 키보드·마우스, 네트워크.
- Windows PC와 microSD 리더를 준비합니다. 예제는 128GB microSD를 사용합니다.
- 기준: JetPack 6.2 / Jetson Linux R36.4.3 / Ubuntu 22.04 / ARM64.
- JetPack 6.2 전용 다운로드 페이지에서 Orin Nano Developer Kit SD card image를 선택합니다.
- 최신 안내가 다른 JetPack 버전으로 바뀔 수 있으므로 6.2 SD 이미지를 명시적으로 선택합니다.

완료: 대상 보드·이미지 버전·기록할 SD 카드의 용량과 드라이브를 확인했습니다.

↗ JetPack 6.2 다운로드 ↗ balenaEtcher 다운로드

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

SD 카드 기록 전에 펌웨어를 확인합니다

1일차 v2 · 부록 / A · 펌웨어 확인

ALL AI

- 이미 JetPack 6.x로 부팅한 장비: 6.2 SD 이미지 기록 절차로 진행합니다.
- 출고 펌웨어 또는 JetPack 5.x 장비: 공식 안내에 따라 QSPI/UEFI 펌웨어를 먼저 갱신합니다.
- SD 카드 이미지를 다시 기록하는 것만으로 보드의 QSPI 갱신이 완료되지는 않습니다.
- 부팅 시 Esc로 UEFI 정보를 확인하고, 불명확하면 아래 공식 갱신 안내에서 장비 상태별 경로를 따릅니다.
- 이후 예제는 JetPack 6.x용 펌웨어 준비가 끝났다는 조건으로 진행합니다.

부팅 실패 시 이미지 반복 기록 전에 펌웨어 호환·전원·DisplayPort 연결을 확인하세요.

↗ JetPack 6.2 설치 안내 ↗ JetPack 6.x 펌웨어 갱신 ↗ NVIDIA Jetson 포럼

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

balenaEtcher로 JetPack 6.2 기록

1일차 v2 · 부록 / A · Windows

ALL AI

- 1. NVIDIA 6.2 페이지에서 SD 이미지를 받고 압축을 풀어 .img 파일을 준비합니다.
- 2. balenaEtcher의 Windows 버전을 설치하고 실행합니다.
- 3. Flash from file → 준비한 JetPack 6.2 .img를 선택합니다.
- 4. Select target → SD 카드의 용량·장치명을 대조하여 선택합니다.
- 5. Flash! → Windows 권한 요청을 승인하고 기록·검증 완료까지 기다립니다.
- 대상 SD 카드의 기존 데이터는 지워집니다. 필요한 파일은 기록 전에 복사합니다.

완료: Etcher의 기록과 검증이 모두 성공했습니다. 단순 파일 복사는 플래싱이 아닙니다.

↗ JetPack 6.2 다운로드 ↗ balenaEtcher 다운로드

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

검증 완료 후 Jetson으로 옮기기

1일차 v2 · 부록 / A → B · 첫 부팅

ALL AI

- Windows에서 포맷 안내가 나타나면 취소합니다. Linux 파티션을 Windows에서 포맷하지 않습니다.
- SD 카드를 안전하게 제거하고 전원이 꺼진 Jetson의 microSD 슬롯에 장착합니다.
- DisplayPort · 키보드 · 마우스 · 네트워크를 연결한 뒤 전원을 연결합니다.
- Ubuntu 초기 설정 화면에서 라이선스, 언어 · 키보드, 시간대, 사용자명 · 암호를 설정합니다.
- 파티션 크기를 묻는 화면이 나오면 사용할 용량을 설정하고 초기 구성이 끝날 때까지 기다립니다.

완료: Ubuntu 데스크톱 로그인과 터미널 실행이 가능합니다.

↗ JetPack 6.2 설치 안내 ↗ balena 포럼 ↗ NVIDIA Jetson 포럼

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

버전과 저장 공간을 먼저 기록

1일차 v2 · 부록 / B · Jetson 호스트

- Jetson 데스크톱에서 Terminal을 열고 다음을 실행합니다.
- Ubuntu는 22.04, 아키텍처는 aarch64/arm64, L4T는 R36 revision 4.3을 기준으로 대조합니다.
- 패키지 갱신 후에는 값이 달라질 수 있으므로 출력값을 그대로 기록합니다.

```
cat /etc/os-release
uname -m
cat /etc/nv_tegra_release
dpkg-query -W nvidia-l4t-core
lsblk -f
df -h /
```

완료: OS·L4T 버전과 루트 저장장치·남은 공간을 확인했습니다.

↗ JetPack 6.2 다운로드

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

네트워크와 SSH 준비

1일차 v2 · 부록 / B · Jetson 호스트

- 유선 또는 Wi-Fi로 인터넷에 연결합니다. 시간대·시각도 맞춥니다.
- 아래는 Jetson 호스트 명령입니다. SSH 설치 후 IP를 확인합니다.

```
sudo apt update
sudo apt install -y curl ca-certificates openssh-server
sudo systemctl enable --now ssh
hostname -I
timedatectl
systemctl is-active ssh
```

완료: Windows PowerShell에서 ssh 사용자명@Jetson-IP로 접속합니다. 사용자명·IP는 실제 값으로 바꿉니다.

↗ NVIDIA Jetson 포럼

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

개발 도구와 JetPack 구성 확인

1일차 v2 · 부록 / B · Jetson 호스트

- 시스템 전체 버전 업그레이드와 필요한 패키지 설치를 구분합니다.
- JetPack 메타패키지가 없으면 현재 L4T 저장소에 맞는 구성 요소를 설치합니다.
- 설치 후 tegrastats로 메모리·온도·GPU 사용률을 관찰하고 Ctrl+C로 끝냅니다.

```
sudo apt install -y git build-essential cmake \
  python3-pip python3-venv v4l-utils
apt-cache policy nvidia-jetpack
sudo apt install nvidia-jetpack
sudo nvpmodel -q
tegrastats
```

완료: 도구 설치·전력 모드·모니터링 확인. GPU 사용률 0%는 유휴 상태일 수 있습니다.

↗ JetPack 6.2 설치 안내

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Docker 1 — 공식 저장소의 키 등록

1일차 v2 · 부록 / C · 독립 절 / Jetson 호스트

- 시작 조건: B 완료, Ubuntu 22.04 ARM64, 인터넷 연결. ROS·LLM 설치는 필요 없습니다.
- Docker가 있으면 docker --version으로 먼저 확인합니다.
- 기존 docker.io·containerd 등과 충돌하면 공식 문서의 충돌 패키지 안내를 확인합니다.

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL \
  https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

다음 장의 저장소 등록을 이어서 실행합니다.

↗ Docker Ubuntu 설치

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Docker 2 — 저장소 등록과 엔진 설치

1일차 v2 · 부록 / C · Jetson 호스트

- 이 저장소 예제는 Ubuntu 22.04 (jammy), ARM64 전용입니다.
- 아래 블록의 EOF까지 한 번에 복사합니다.

```
sudo tee /etc/apt/sources.list.d/docker.sources <<'EOF'
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: jammy
Components: stable
Architectures: arm64
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

완료: Docker Engine 패키지 설치가 오류 없이 끝났습니다.

↗ Docker Ubuntu 설치

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

Docker 3 — 권한과 단독 실행 확인

1일차 v2 · 부록 / C · Jetson 호스트

- docker 그룹은 관리자 수준의 접근 권한을 제공합니다. 개인 실습 계정에만 적용합니다.
- usermod 이후 데스크톱 로그아웃·재로그인 또는 SSH 재접속을 수행합니다.

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"

# After logout / login, in a NEW terminal:
id -nG
docker version
docker run --rm hello-world
```

완료: sudo 없이 Hello from Docker! 출력. 이 검사는 Docker 자체 확인이며 GPU 검사는 D에서 합니다.

↗ Docker Ubuntu 설치

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

L4T 1 — NVIDIA 런타임 준비

1일차 v2 · 부록 / D · 독립 절 / Jetson 호스트

- 시작 조건: B·C 완료. ROS·LLM 설치는 필요 없습니다.
- JetPack 저장소에서 Toolkit을 설치하고 Docker 설정을 갱신합니다.
- Docker 재시작은 실행 중인 컨테이너에 영향을 줄 수 있으므로 작업을 저장합니다.

```
sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
docker info --format '{{json .Runtimes}}'
```

완료: 출력에 nvidia 런타임이 있습니다. 패키지 후보가 없으면 JetPack 저장소·공식 설치 안내를 확인합니다.

↗ JetPack 6.2 설치 안내 ↗ NVIDIA Container Toolkit

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

L4T 2 — 이미지 선택과 다운로드

1일차 v2 · 부록 / D · Jetson 호스트

- NGC의 NVIDIA L4T JetPack → Tags에서 공개 태그와 설명을 확인합니다.
- 예제 태그 r36.4.0은 실제 공개 태그입니다. 호스트 6.2의 L4T R36.4.3과 숫자가 다릅니다.
- 아래는 다운로드 예제이며, 이 조합의 GPU 동작은 다음 장에서 별도 검증합니다.
- 호스트 버전으로 이미지 태그를 임의 생성하지 않습니다. 강의 ROS·Jupyter는 이 이미지에 자동 포함되지 않습니다.

```
docker pull nvcr.io/nvidia/l4t-jetpack:r36.4.0
docker image inspect nvcr.io/nvidia/l4t-jetpack:r36.4.0 \
  --format '{{.Architecture}} {{json .RepoDigests}}'
```

완료: pull 성공, arm64 및 digest 기록. manifest unknown이면 NGC의 실제 태그를 다시 확인합니다.

↗ NGC L4T JetPack

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

L4T 3 — 실행과 파일 저장

1일차 v2 · 부록 / D · Jetson 호스트 → 컨테이너

- Jetson 호스트의 ~/jetson-lab을 컨테이너 /work에 연결합니다.
- 실행 후 표시되는 셸은 컨테이너 내부입니다. 뒤의 GPU 검사를 이 셸에서 진행합니다.

```
mkdir -p "$HOME/jetson-lab"
docker run --rm -it --runtime=nvidia \
  --name l4t-lab \
  -v "$HOME/jetson-lab:/work" \
  -w /work \
  nvcr.io/nvidia/l4t-jetpack:r36.4.0 bash

# Inside container:
cat /etc/os-release
nvcc --version
```

nvcc 출력은 도구 설치 확인입니다. 실제 GPU 접근 성공 여부는 다음 검사로 확인합니다.

↗ NGC L4T JetPack ↗ NVIDIA Container Toolkit

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

L4T 4 — CUDA 장치 접근 검사

1일차 v2 · 부록 / D · 컨테이너 내부

- 컨테이너 셸에서 아래 블록을 실행합니다. count가 1 이상이고 status=0이어야 합니다.
- 실패하면 호스트 L4T·런타임·이미지 태그와 오류 출력을 함께 기록합니다.

```
cat > /tmp/gpu-check.cu <<'EOF'
#include <cuda_runtime.h>
#include <stdio.h>
int main() {
  int n = 0;
  cudaError_t e = cudaGetDeviceCount(&n);
  printf("status=%d count=%d %s\n", (int)e, n,
         cudaGetErrorString(e));
  return (e == cudaSuccess && n > 0) ? 0 : 1;
}
EOF
nvcc /tmp/gpu-check.cu -o /tmp/gpu-check
/tmp/gpu-check
```

완료: CUDA 장치 열거 성공. 전체 AI 워크로드 성능·호환성 검증은 별도입니다.

↗ NGC L4T JetPack ↗ NVIDIA Jetson 포럼

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

L4T 5 — 종료와 재실행

1일차 v2 · 부록 / D · 컨테이너 → Jetson 호스트

- 컨테이너 내부에서 exit하면 --rm 설정으로 컨테이너가 삭제됩니다.
- 다운로드한 이미지는 남고, /work에 저장한 파일은 호스트 ~/jetson-lab에 남습니다.
- 다음 실행은 앞의 docker run 명령을 다시 사용합니다. 삭제된 컨테이너에는 docker start를 쓰지 않습니다.

```
# Inside container:
exit

# Jetson host:
docker ps -a
docker image ls
ls "$HOME/jetson-lab"
```

D 완료: 이미지 다운로드·컨테이너 실행·CUDA 장치 확인·저장 경로를 구분했습니다.

↗ Docker Ubuntu 설치 ↗ NGC L4T JetPack

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

ROS 2 1 — Humble 설치 준비

1일차 v2 · 부록 / E · 독립 절 / Jetson 호스트

- 시작 조건: B 완료. Docker·L4T 이미지 절을 건너뛰어도 됩니다.
- 컨테이너 밖의 Jetson 터미널에서 실행합니다. Ubuntu 22.04·arm64를 확인합니다.
- ROS는 시스템 Python을 사용합니다. LLM 가상환경이 켜져 있으면 새 터미널에서 진행합니다.

```
dpkg --print-architecture
. /etc/os-release; echo "$VERSION_CODENAME"
sudo apt update
sudo apt install -y locales software-properties-common curl
sudo locale-gen en_US.UTF-8
export LANG=en_US.UTF-8
sudo add-apt-repository -y universe
```

완료: arm64 / jammy, UTF-8 locale. 다른 Ubuntu 버전에는 이 절차를 그대로 적용하지 않습니다.

↗ ROS 2 Humble 설치 ↗ ROS 공식 설치 문서 원본

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

ROS 2 2 — 공식 저장소 설정 패키지

1일차 v2 · 부록 / E · Jetson 호스트

- 공식 ros-apt-source 릴리스에서 Jammy용 패키지를 받아 저장소·키를 등록합니다.
- 아래 명령은 한 Bash 터미널에서 순서대로 실행합니다. 네트워크 오류가 나면 중단합니다.

```
ROS_APT_VER=$(curl -fsSL \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["tag_name"])')
test -n "$ROS_APT_VER"
ROS_APT_BASE=https://github.com/ros-infrastructure/ros-apt-source
curl -fL -o /tmp/ros2-apt-source.deb \
  "$ROS_APT_BASE/releases/download/$ROS_APT_VER/ros2-apt-source_${ROS_APT_VER}.jammy_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
sudo apt update
```

완료: apt update에 ROS 저장소가 나타나고 서명·다운로드 오류가 없습니다.

↗ ROS 저장소 설정 원본

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

ROS 2 3 — 설치와 새 터미널 준비

1일차 v2 · 부록 / E · Jetson 호스트

- 먼저 설치 계획을 확인합니다. 핵심 시스템 패키지 제거가 제안되면 진행하지 않고 공식 설치 주의사항을 확인합니다.
- 초기 Ubuntu의 systemd·udev 갱신 주의사항과 JetPack 버전 유지 조건을 함께 확인합니다.

```
sudo apt-get -s install ros-humble-desktop ros-dev-tools
sudo apt install ros-humble-desktop ros-dev-tools
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=21
ros2 --help
```

완료: ros2 도움말 출력. 새 터미널마다 source와 Domain ID를 적용합니다. 21은 개인 배정값 예시입니다.

↗ ROS 2 Humble 설치 ↗ ROS 공식 설치 문서 원본

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

ROS 2 4 — talker와 listener 확인

1일차 v2 · 부록 / E · Jetson 호스트 / 터미널 2개

- 아래 A·B를 서로 다른 터미널에서 실행합니다. 두 터미널의 Domain ID는 같아야 합니다.
- 한 Jetson을 함께 쓰는 경우 다른 사용자의 실습과 Domain ID를 구분합니다.

```
# Terminal A:
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=21
ros2 run demo_nodes_cpp talker

# Terminal B:
source /opt/ros/humble/setup.bash
export ROS_DOMAIN_ID=21
ros2 run demo_nodes_py listener
```

완료: A의 Publishing 메시지가 B의 I heard로 수신됩니다. 각 터미널에서 Ctrl+C로 종료합니다.

↗ ROS 2 Humble 설치

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

ROS 2 5 — GUI와 workspace 준비

1일차 v2 · 부록 / E · Jetson 호스트

- turtlesim은 Jetson 데스크톱 터미널에서 실행하고 창을 확인한 뒤 Ctrl+C로 종료합니다.
- 그다음 기본 ament_python 패키지를 만들고 colcon 빌드·등록을 확인합니다.

```
source /opt/ros/humble/setup.bash
ros2 run turtlesim turtlesim_node
# Stop turtlesim with Ctrl+C before continuing.
mkdir -p "$HOME/ros2_ws/src"
cd "$HOME/ros2_ws/src"
ros2 pkg create --build-type ament_python my_first_pkg
cd "$HOME/ros2_ws"
colcon build --symlink-install
source install/setup.bash
ros2 pkg prefix my_first_pkg
```

E 완료: 통신·GUI·빌드 성공. 교재 nepes_ros_lab은 별도 소스·의존성 설치가 필요합니다.

↗ ROS 2 Humble 설치 ↗ Robotics Stack Exchange

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

LLM 1 — Ollama 설치와 서비스

1일차 v2 · 부록 / F · 독립 절 / Jetson 호스트

- 시작 조건: B 완료. Docker·ROS는 필요 없습니다. 호스트 직접 설치 예제입니다.
- 먼저 11434 포트를 확인합니다. 기존 Ollama가 있으면 중복 서버를 설치·실행하지 않습니다.
- 설치 스크립트는 공식 사이트에서 내려받아 내용을 확인한 뒤 실행합니다.

```
ss -ltnp | grep ':11434'
curl -fsSL https://ollama.com/install.sh -o /tmp/ollama-install.sh
less /tmp/ollama-install.sh
# Press q to leave less, then install:
sh /tmp/ollama-install.sh
sudo systemctl enable --now ollama
ollama --version
systemctl is-active ollama
```

완료: active 및 버전 출력. 최신 설치본의 JetPack 6.2 호환성은 실장비에서 확인해야 합니다.

↗ Ollama Linux 설치 ↗ Ollama 문제 해결

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

LLM 2 — 작은 모델로 추론 확인

1일차 v2 · 부록 / F · Jetson 호스트

- 첫 예제는 gemma3:1b입니다. 최초 pull에는 인터넷과 저장 공간이 필요합니다.
- 답변이 나온 뒤 ollama ps와 로그를 보고 GPU 사용 여부를 확인합니다.
- CPU 응답 성공과 GPU 추론 성공을 구분해서 기록합니다.

```
ollama pull gemma3:1b
ollama run gemma3:1b "Explain ROS 2 in one sentence."
ollama ps
curl -fsS http://localhost:11434/api/tags
journalctl -u ollama -n 40 --no-pager
```

완료: 텍스트 응답·모델 목록 확인. GPU 미사용이면 버전·서비스 로그로 검색하고 실행 결과를 기록합니다.

↗ Ollama Linux 설치 ↗ Ollama 문제 해결

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

LLM 3 — Python 가상환경과 Jupyter

1일차 v2 · 부록 / F · Jetson 호스트

- LLM 도구는 별도 가상환경에 설치합니다. ROS의 시스템 Python과 분리합니다.
- 서버와 Python은 별개입니다. Ollama 서비스가 실행 중이어야 다음 장의 API 호출이 됩니다.

```
python3 -m venv "$HOME/venvs/llm"
source "$HOME/venvs/llm/bin/activate"
python -m pip install requests jupyterlab ipykernel langchain-ollama
python -m pip check
python -m ipykernel install --user --name jetson-llm \
  --display-name "Python (Jetson LLM)"
python -m pip freeze > "$HOME/jetson-llm-requirements.txt"
jupyter lab --ip=127.0.0.1
```

완료: Jetson 브라우저에서 출력 URL·토큰으로 접속 후 Python (Jetson LLM) 커널을 선택합니다.

↗ Ollama Linux 설치

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

LLM 4 — Python에서 호스트 API 호출

1일차 v2 · 부록 / F · Jupyter Python 셀

- 앞에서 등록한 커널을 사용합니다. 이 블록은 Bash 명령이 아니라 Python 코드입니다.
- 응답이 오면 호스트 Ollama → HTTP API → Python 연결이 완료된 것입니다.

```
import sys
import requests
print(sys.executable)
response = requests.post(
    "http://127.0.0.1:11434/api/generate",
    json={"model": "gemma3:1b",
          "prompt": "Explain ROS 2 in one sentence.",
          "stream": False},
    timeout=180,
)
response.raise_for_status()
print(response.json()["response"])
```

F 완료: 가상환경 경로와 답변 확인. Connection refused는 서비스·포트, 모델 오류는 ollama list를 확인합니다.

↗ Ollama 문제 해결

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

경로별 완료 기준을 확인합니다

1일차 v2 · 부록 / G · 완료 점검

ALL AI

- A·B: JetPack 6.2 부팅 / OS·L4T 기록 / 네트워크·SSH / 저장 공간 확인.
- C: Docker 서비스 active / 일반 사용자 hello-world 성공.
- D: L4T 이미지 pull·digest 기록 / 컨테이너 실행 / CUDA count ≥ 1.
- E: 호스트 ROS talker-listener 수신 / turtlesim 창 / colcon 빌드.
- F: Ollama 답변 / GPU 사용 여부 기록 / 가상환경 Python API 응답.
- 실행 일시·버전·정상 출력·실패 로그를 남깁니다. 설치 완료와 GPU 검증 완료를 각각 표시합니다.

이 자료는 명령·구성 검토 완료 단계입니다. 위 실장비 완료 항목은 사용자가 직접 확인해야 합니다.

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

재설치 전에 원인을 좁힙니다

1일차 v2 · 부록 / G · 오류 해결

ALL AI

- Etcher 검증 실패 → SD · 리더 · 이미지 파일. 부팅 실패 → QSPI · 전원 · 화면 연결.
- Docker permission denied → 재로그인 · 그룹. runtime 오류 → Toolkit · Docker 설정.
- L4T manifest unknown → NGC 실제 태그. CUDA 오류 → 호스트 · 이미지 버전과 런타임.
- ros2 없음 → source · 설치. 메시지 없음 → Domain ID · 노드 · 네트워크.
- Ollama 연결 실패 → 서비스 · 11434 포트. 메모리 부족 → 모델 크기 · 동시 실행 수.
- 검색 시 장비명 + 실제 버전 + 오류 핵심 문장을 사용하고 해결법의 적용 버전을 확인합니다.

보조 문서: 운영자료/1일차_v2_부록/설치안내.md — 복사용 명령과 전체 출처

↗ NVIDIA Jetson 포럼 ↗ balena 포럼 ↗ Docker Ubuntu 설치 ↗ Robotics Stack Exchange ↗ Ollama 문제 해결

본 자료의 저작권은 (주)올에이아이에 있으며, 사전 승인 없이
무단 전재, 복사, 배포, 수정을 엄격히 금합니다.

감사합니다

경기도 성남시 분당구 판교로 289번길 20 판교테크노밸리
스타트업캠퍼스 1동 806호-807호

Copyright @ 2026 ALL.AI Inc. All rights reserved.

www.allai.co.kr